

University of Idaho
College of Engineering

CS 466/566
PLC PROGRAMMING AND AUTOMATION
FALL 2024

LECTURE 5 – DIGITAL LOGIC II

INSTRUCTOR: DR JOHN SHOVIC
LECTURER: JACOB FRIEDBERG

DIGITAL LOGIC 2

- Boolean Logic
  - Logic Gates
  - Truth Tables
  - Combinational Logic
  - Theorems
  - Simplification
  - Building blocks
- Sequential Logic
  - Latches, flip flops
  - State
  - State machines

LOGIC OPERATIONS

LOGICAL OPERATIONS CAN BE DONE GIVEN 1 OR MORE INPUTS

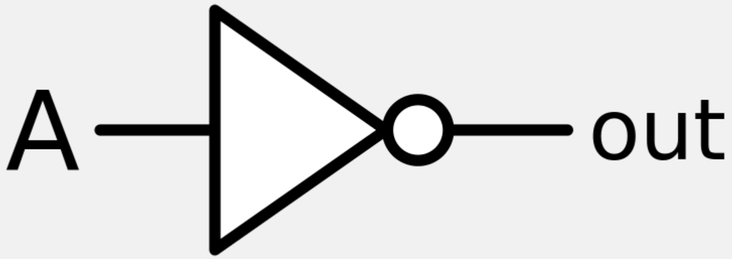
- Truth tables show all possible combinations and outcomes
- Logic gates are a graphical representation
- Inputs usually called A, B, C, D
- Outputs usually called X, Y, Z, Q, Out – no hard rule on these
- NOT, AND, OR are the basic logic operations – all other operations can be created from these
- Logical operator precedence goes NOT > AND > OR

LOGIC OPERATIONS

NOT – OUTPUT IS THE OPPOSITE OF THE INPUT - 1-> 0, 0->1

- Also called an Inverter
- Can be notated as ! A or $\bar{A}$ - will use the latter in examples
- In programming, NOT is usually notated as !
- Can be simplified to a circle if attached to another gate input or output

A	Out
0	1
1	0

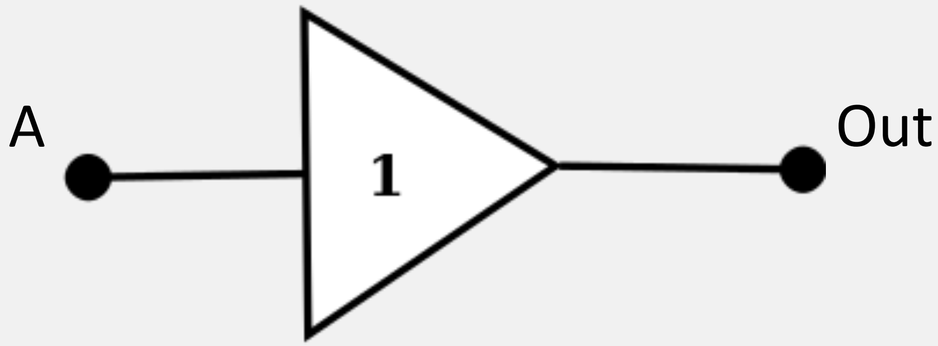


LOGIC OPERATIONS

BUFFER

- What goes in, also goes out
- Used to help with timing, isolating input from output

A	Out
0	0
1	1

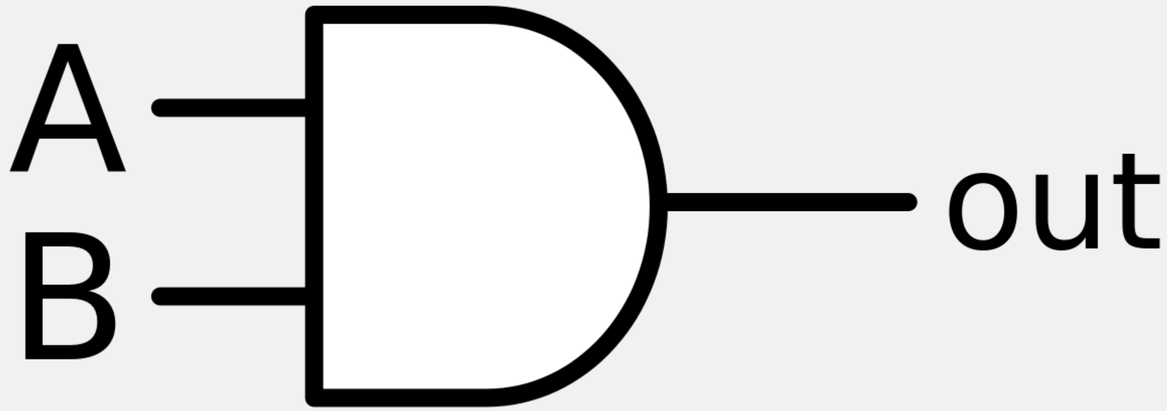


LOGIC OPERATIONS

- AND – Output is true if and only if all inputs are true

- Notated as AB or $A \cdot B$
- Programming is usually notated as &&

A	B	Out
0	0	0
0	1	0
1	0	0
1	1	1

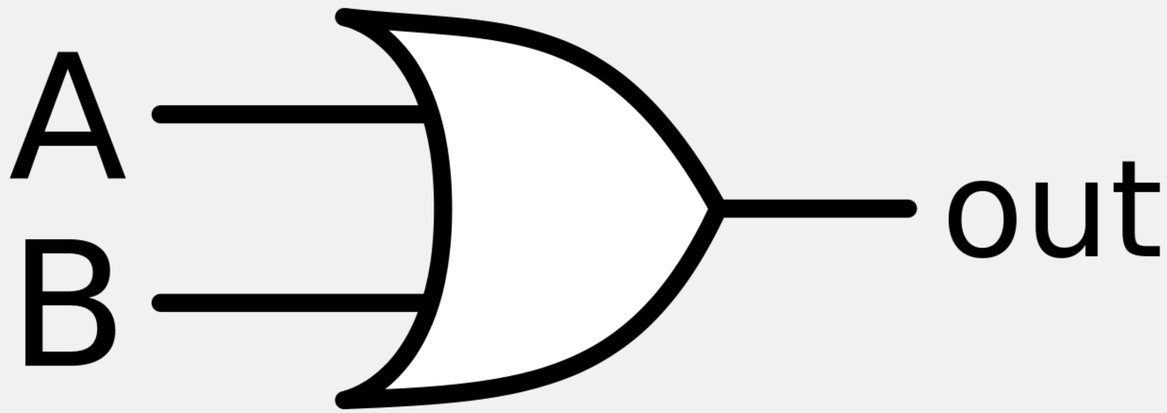


LOGIC OPERATIONS

- OR – Output is true if any input is true

- Notated as $A + B$
- Programming is usually notated as ||

A	B	Out
0	0	0
0	1	1
1	0	1
1	1	1



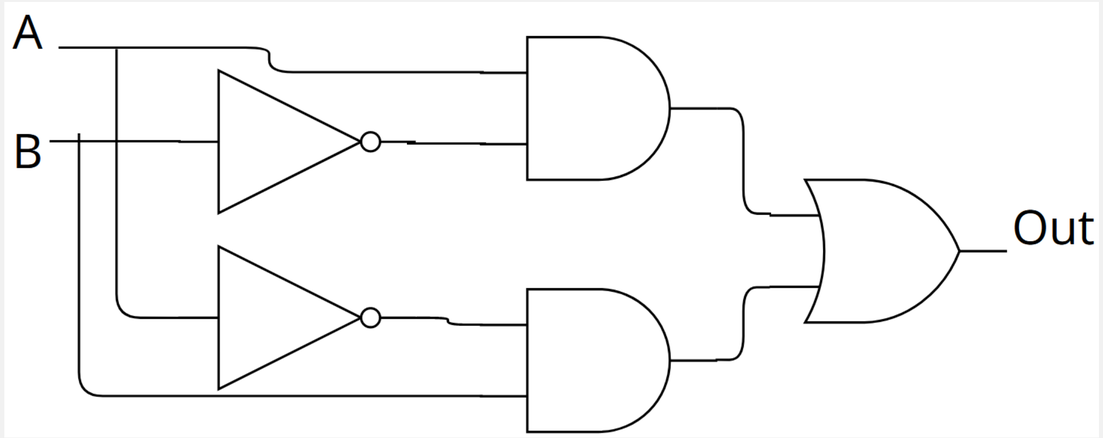
LOGIC OPERATIONS

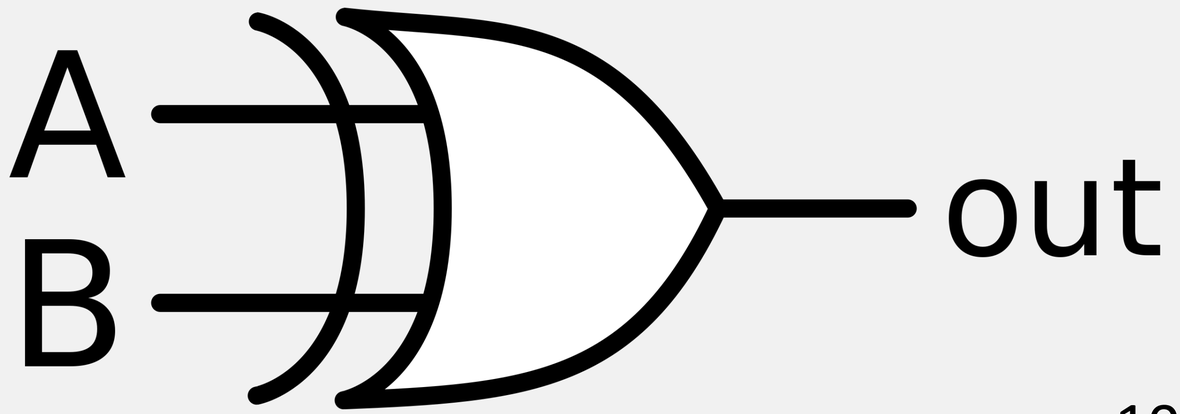
- XOR – (Exclusive Or) Output is true iff one input is true

  Alternatively true if odd number of inputs are true for >2 input XOR

- Notated as $A \oplus B$ *or* $A \cdot \bar{B} + \bar{A} \cdot B$

A	B	Out
0	0	0
0	1	1
1	0	1
1	1	0



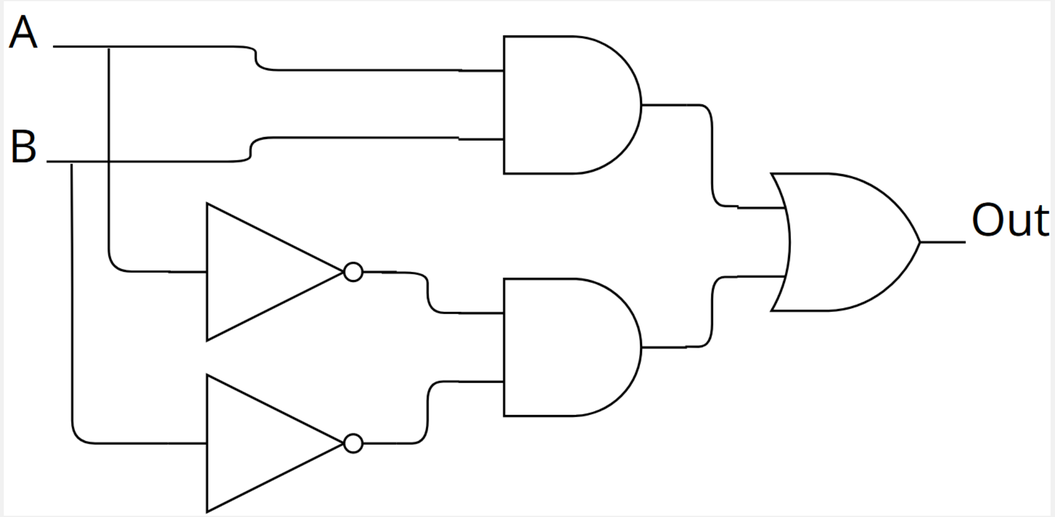


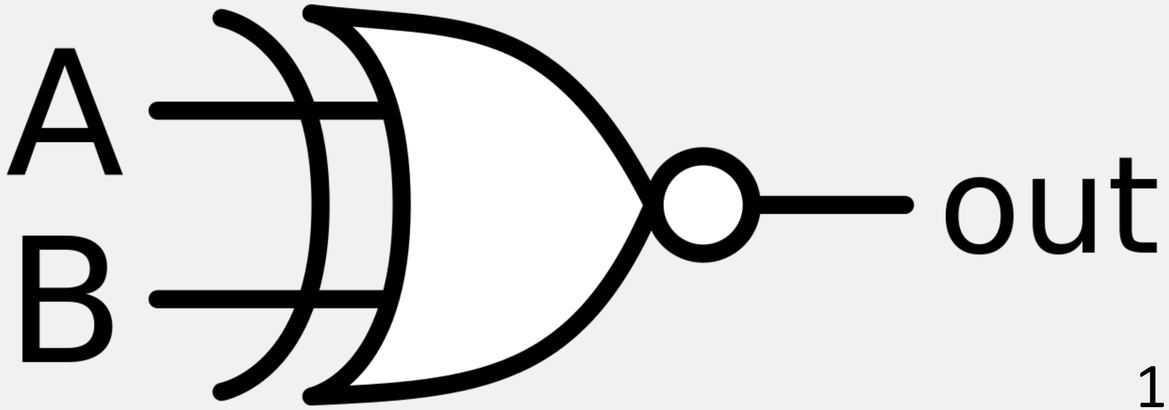
LOGIC OPERATIONS

- XNOR – (Exclusive Not OR) complement of XOR where output is true if both inputs are false, or both inputs are true

- Notated as $\overline{A \oplus B}$ or $A \cdot B + \bar{A} \cdot \bar{B}$

A	B	Out
0	0	1
0	1	0
1	0	0
1	1	1

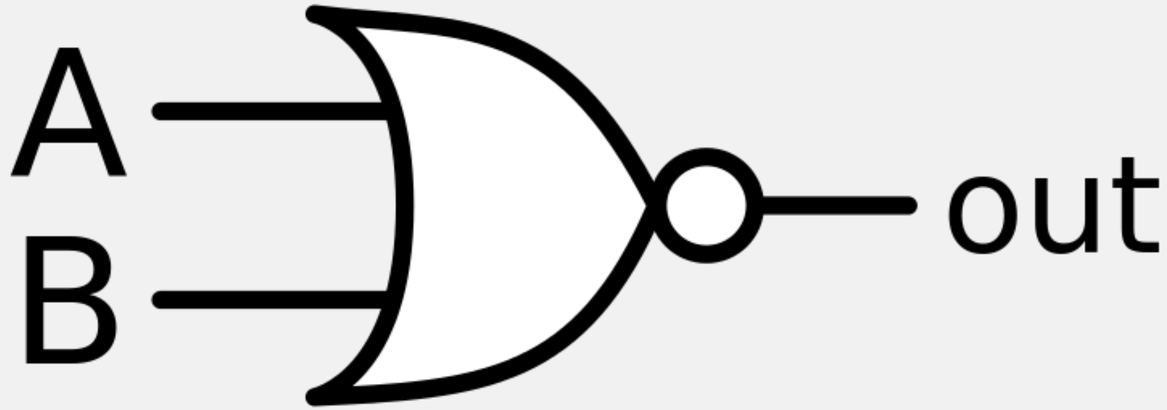




LOGIC OPERATIONS

- NOR – (Not OR) complement of OR where output is true if neither input is true
- A universal gate – all other gates can be created from a NOR gate
  - Notated as $\overline{A+B}$

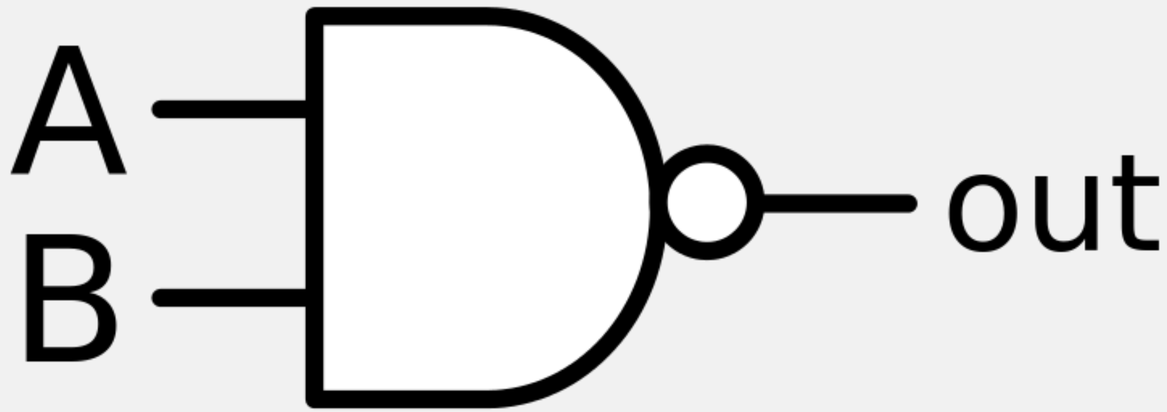
A	B	Out
0	0	1
0	1	0
1	0	0
1	1	0



LOGIC OPERATIONS

- NAND – (Not AND) complement of AND where output is true if any input is false
- Also a universal gate
  - Notated as $\overline{AB}$ or $\overline{A \cdot B}$

A	B	Out
0	0	1
0	1	1
1	0	1
1	1	0



PROJECT 2 WEDNESDAY

- Please turn in on canvas a zip file containing your factoryIO Scene and the OpenPLC Code folder.
- We will present Project 2 starting Wednesday during class.
  - Short 5 minute summary of what your simple factory does, and any interesting things you learned or challenging parts to program.

TRUTH TABLES

- Any number of inputs can be combined to produce an output
  - Use to show what inputs make an output true or false
- In combinational logic, there is no memory
- Output depends only on the current state of the inputs
- Break down an expression into its simplest components, then recombine them using the logical operations
- Can translate to gate representation

TRUTH TABLES

- Examples – Do the truth table and gate representation
  - $Y = A \cdot B + \bar{A} \cdot B \cdot C$
  - $Y = \bar{A} \cdot B \cdot \bar{C} + A \cdot B + \overline{A \cdot C}$

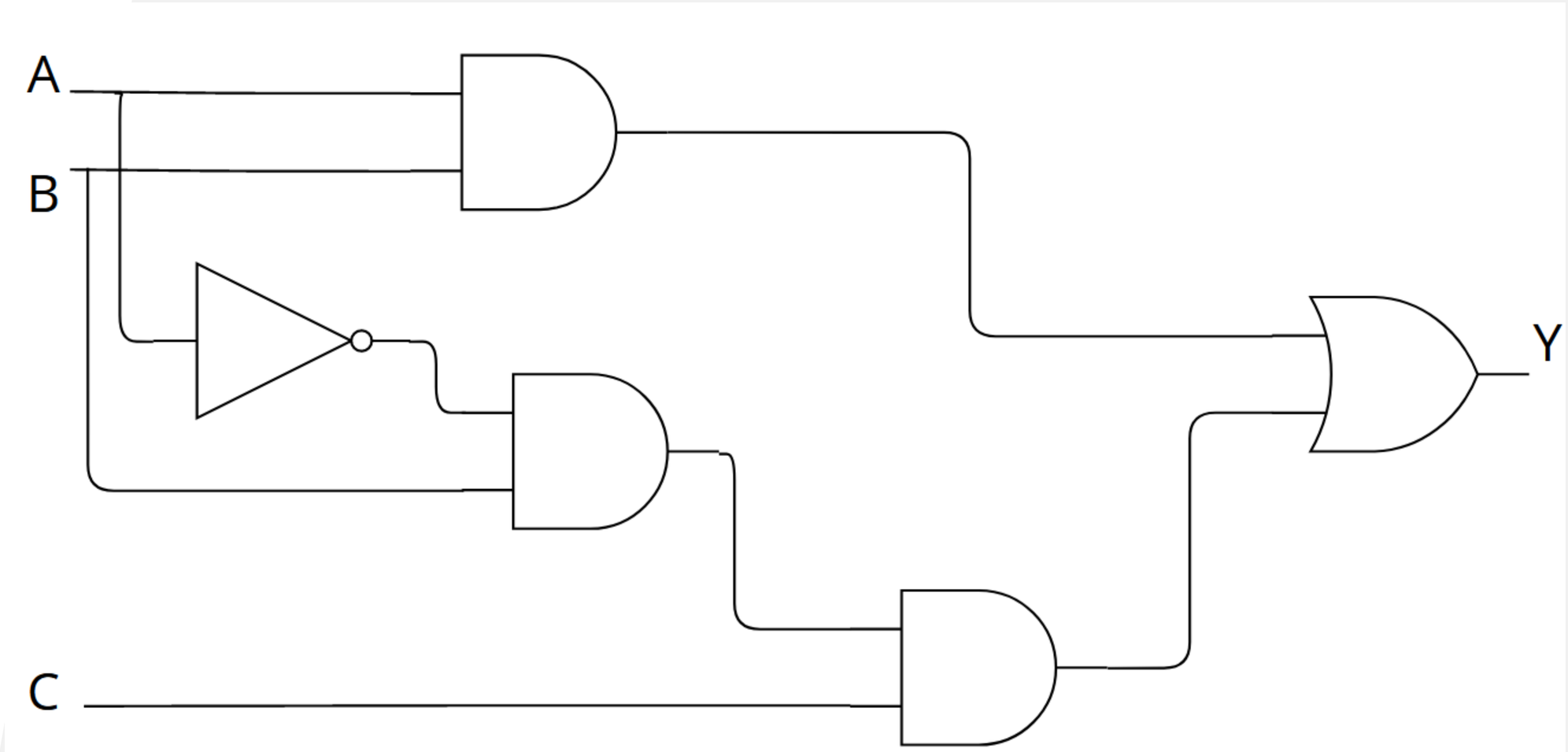
TRUTH TABLES

- $Y = A \cdot B + \bar{A} \cdot B \cdot C$

A	B	C	$A \cdot B$	$\bar{A}$	$\bar{A} \cdot B$	$\bar{A} \cdot B \cdot C$	$A \cdot B + \bar{A} \cdot B \cdot C$
0	0	0	0	1	0	0	0
0	0	1	0	1	0	0	0
0	1	0	0	1	1	0	0
0	1	1	0	1	1	1	1
1	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0
1	1	0	1	0	0	0	1
1	1	1	1	0	0	0	1

GATE REPRESENTATION

- $Y = A \cdot B + \bar{A} \cdot B \cdot C$



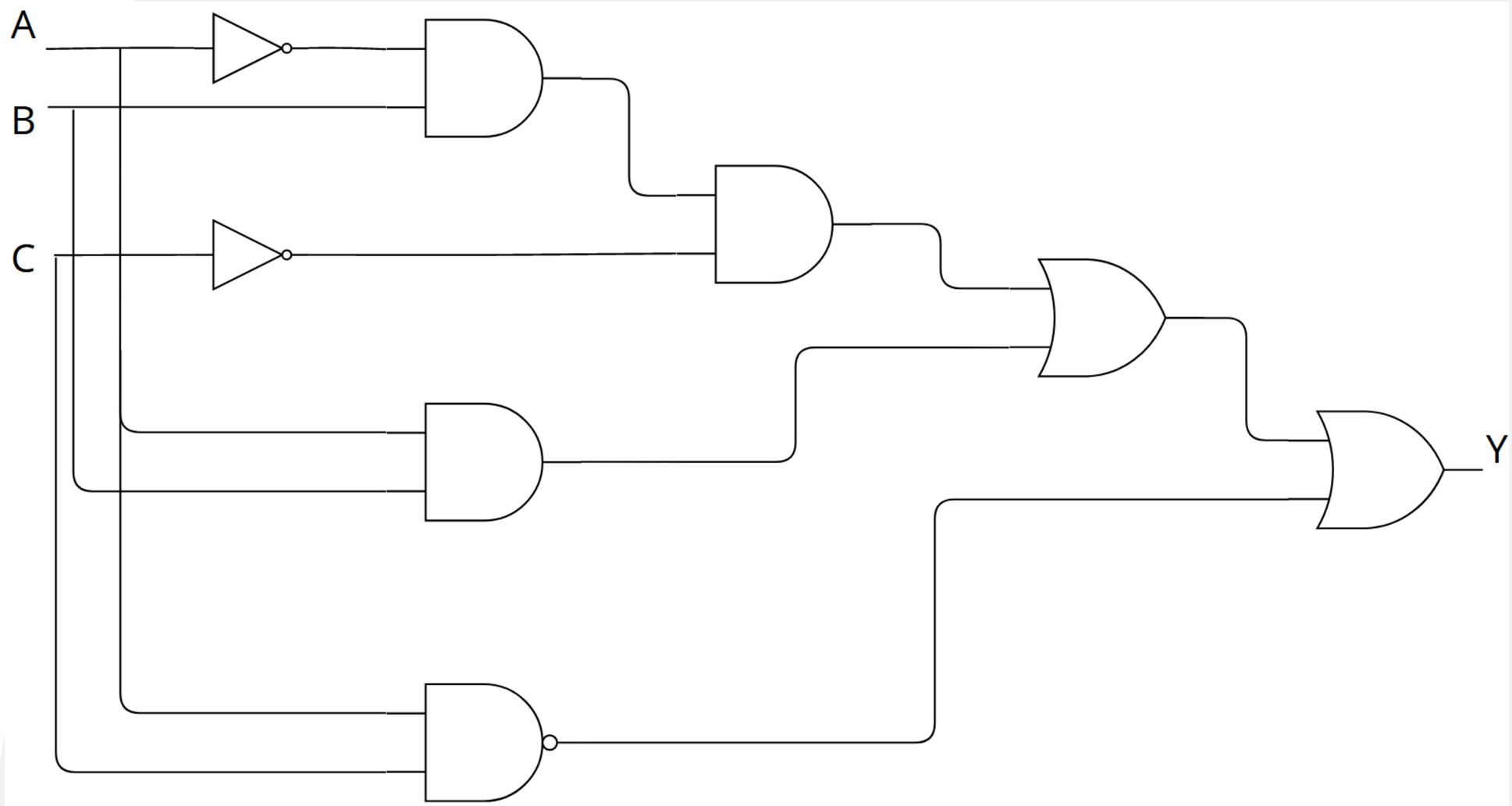
TRUTH TABLES

- $Y = \bar{A} \cdot B \cdot \bar{C} + A \cdot B + \overline{A \cdot C}$

A	B	C	$\bar{A}$	$\bar{A} \cdot B$	$\bar{C}$	$\bar{A} \cdot B \cdot \bar{C}$	$A \cdot B$	$A \cdot C$	$\overline{A \cdot C}$	$\bar{A} \cdot B \cdot \bar{C} + A \cdot B + \overline{A \cdot C}$
0	0	0	1	0	1	0	0	0	1	1
0	0	1	1	0	0	0	0	0	1	1
0	1	0	1	1	1	1	0	0	1	1
0	1	1	1	1	0	0	0	0	1	1
1	0	0	0	0	1	0	0	0	1	1
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	1	0	1	0	1	1
1	1	1	0	0	0	0	1	1	0	1

GATE REPRESENTATION

- $Y = \bar{A} \cdot B \cdot \bar{C} + A \cdot B + \overline{A \cdot C}$



OVERFLOW AND UNDERFLOW REVISITED

DETECT OVERFLOW LOGICALLY

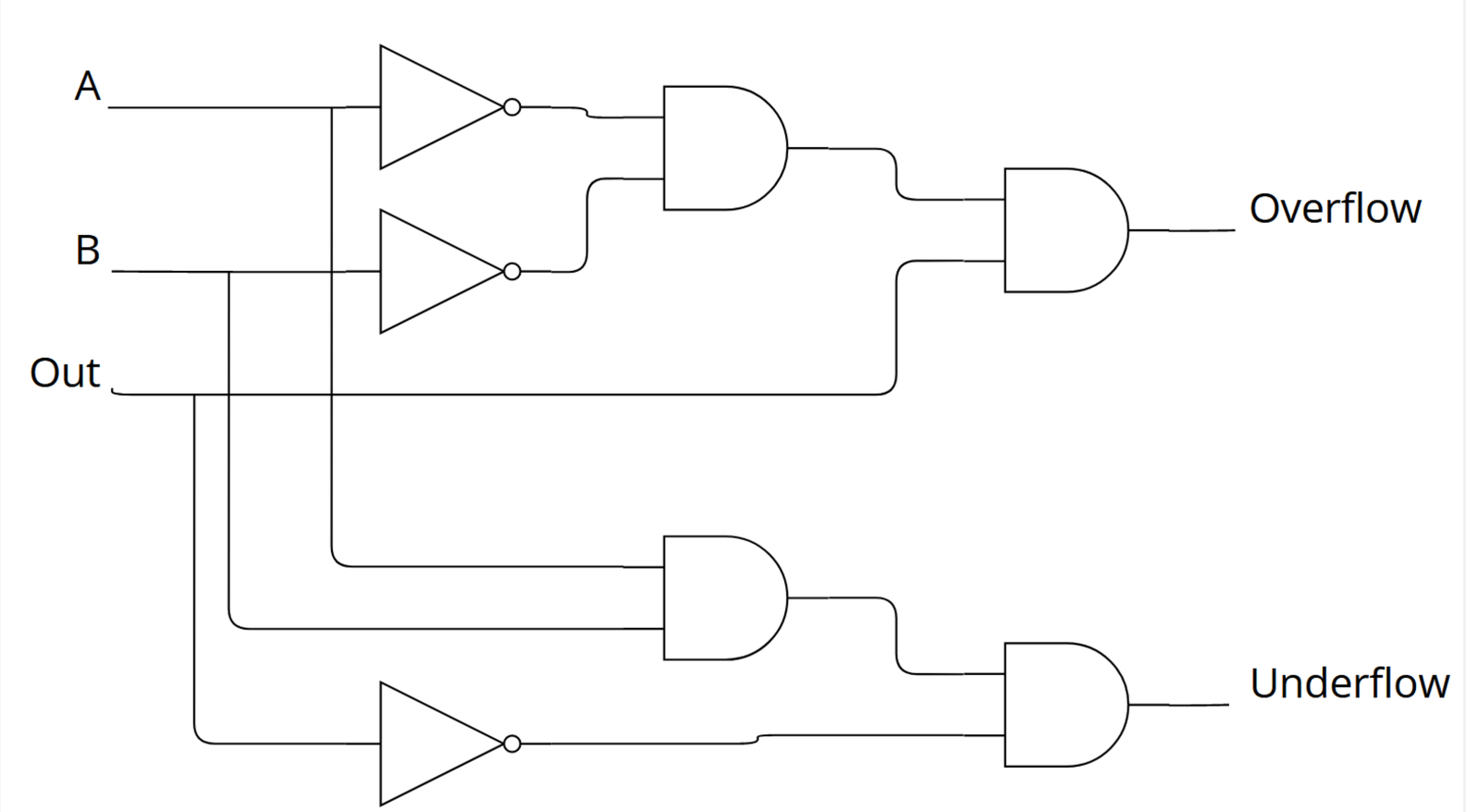
- Inputs – MSB of Input A, Input B, MSB of Output
- Output – Overflow Occurred, Underflow Occurred

Inputs			Outputs	
MSB A	MSB B	MSB Output	Overflow	Underflow
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	0
1	0	0	0	0
1	0	1	0	0
1	1	0	0	1
1	1	1	0	0

OVERFLOW AND UNDERFLOW REVISITED

DETECT OVERFLOW LOGICALLY

- Overflow and underflow each only occur in 1 specific case
- $Overflow = \bar{A} \cdot \bar{B} \cdot Out$
- $Underflow = A \cdot B \cdot \overline{Out}$
- Draw a circuit using gates



OVERFLOW AND UNDERFLOW

DETECT OVERFLOW LOGICALLY

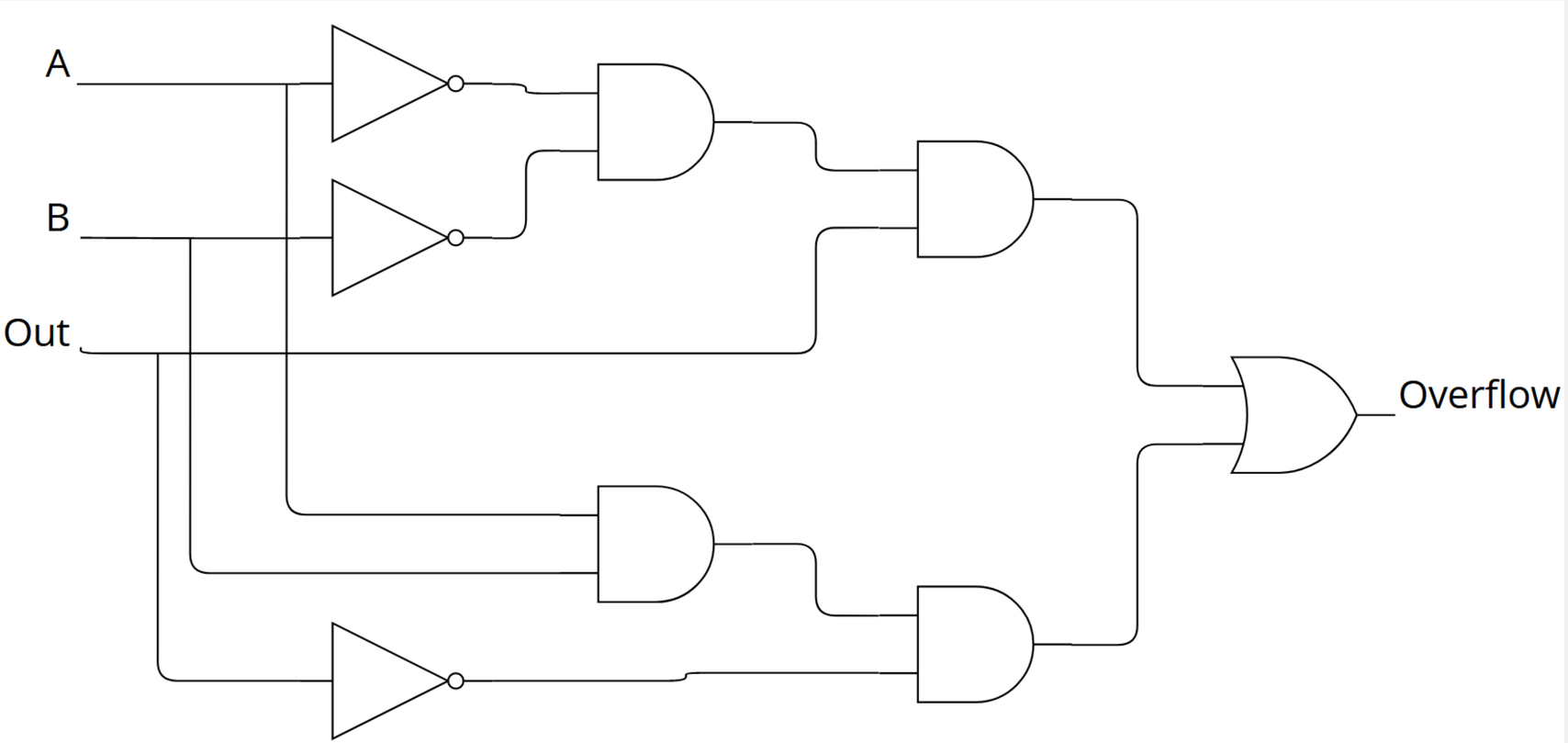
- 2 Outputs is too expensive/not enough space/not enough pins
- Use only 1 output as a general overflow/underflow detector

Inputs			Outputs
MSB A	MSB B	MSB Output	Overflow (combined)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

OVERFLOW AND UNDERFLOW REVISITED

DETECT OVERFLOW LOGICALLY

- $Overflow = \bar{A} \cdot \bar{B} \cdot Out + A \cdot B \cdot \overline{Out}$
- Draw the circuit using gates
- Cost one more gate
- Only one output



BOOLEAN LOGIC THEOREMS

Theorem	Dual	Name
$A \cdot 0 = 0$	$A + 1 = 1$	Annulment
$A \cdot 1 = A$	$A + 0 = A$	Identity
$A \cdot 0 = 0$	$A + 1 = 1$	Identity
$A \cdot A = A$	$A + A = A$	Idempotent
$A \cdot \bar{A} = 0$	$A + \bar{A} = 1$	Complement
$\bar{\bar{A}} =$A		Double Negation
$A \cdot B = B \cdot A$	$\mathrm{A} + B = B + A$	Commutativity
$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	$A + (B + C) = (A + B) + C$	Associativity
$(A \cdot B) + (A \cdot C) = A \cdot (B + C)$	$A + B \cdot C = (A + B) \cdot (A + C)$	Distributivity

BOOLEAN LOGIC THEOREMS

Theorem	Dual	Name
$(A+B)\cdot(A+\bar{B})=A$	$(A\cdot B)+(A\cdot\bar{B})=A$	Redundancy a
$(A+\bar{B})\cdot B=A\cdot B$	$(A\cdot\bar{B})+B=A+B$	Redundancy b
$A\cdot(A+B)=A$	$A+A\cdot B=A$	Absorption
$(A\cdot B)+(A\cdot\bar{B})=A$	$(A+B)\cdot(A+\bar{B})=A$	Combining
$A\cdot(A+B)=A$	$A+(A\cdot B)=A$	Covering
$(A+B)\cdot(\bar{A}+C)\cdot(B+C)$ $=(A+B)(\bar{A}+C)$	$A\cdot B+(\bar{A}\cdot C)+(B\cdot C)$ $=(AB)+(\bar{A}\cdot C)$	Consensus
$\overline{A\cdot B}=\bar{A}+\bar{B}$	$\overline{A+B}=\bar{A}\cdot\bar{B}$	De Morgan's
$A\bigoplus B=(A+\bar{B})\cdot(\bar{A}+B)$	$A\bigoplus B=(A\cdot\bar{B})+(\bar{A}\cdot B)$	XOR
$\overline{A\bigoplus B}=(A+B)\cdot(\overline{A\cdot B})$	$\overline{A\bigoplus B}=(\bar{A}\cdot\bar{B})+(A\cdot B)$	XNOR
$\overline{A\bigoplus B}=(A+B)\cdot(\bar{A}+\bar{B})$		XNOR

TRUTH TABLES OF MORE COMPLEX THEOREMS

- Redundancy a – Combination of Distribution, Complement and Idempotent
- $(A + B) \cdot (A + \bar{B}) = A$

A	B	$A + B$	$\bar{B}$	$A + \bar{B}$	$(A + B) \cdot (A + \bar{B})$	A
0	0	0	1	1	0	0
0	1	1	0	0	0	0
1	0	1	1	1	1	1
1	1	1	0	1	1	1

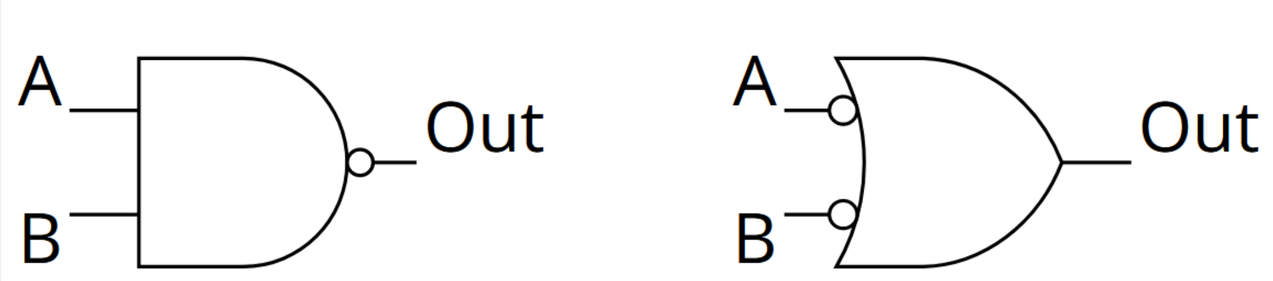
TRUTH TABLES OF MORE COMPLEX THEOREMS

- Redundancy b – Combination of Distribution, Complement and Idempotent
- $(A \cdot B) + (A \cdot \bar{B}) = A$

A	B	$A \cdot B$	$\bar{B}$	$A \cdot \bar{B}$	$(A \cdot B) + (A \cdot \bar{B})$	A
0	0	0	1	0	0	0
0	1	0	0	0	0	0
1	0	0	1	1	1	1
1	1	1	0	0	1	1

DE MORGAN'S LAW

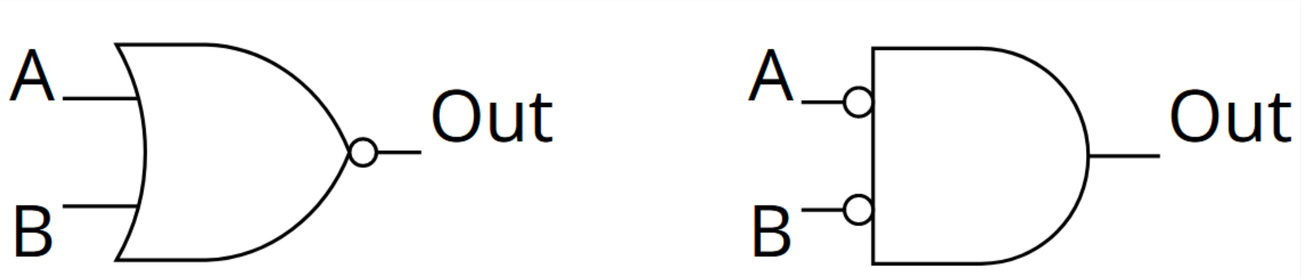
- $\overline{A \cdot B} = \bar{A} + \bar{B}$
- A NAND gate is equivalent to an OR gate with inverted inputs



A	B	$A \cdot B$	$\overline{A}$	$\overline{B}$	$\overline{A \cdot B}$	$\overline{A} + \overline{B}$
0	0	0	1	1	1	1
0	1	0	1	0	1	1
1	0	0	0	1	1	1
1	1	1	0	0	0	0

DE MORGAN'S LAW

- $\overline{A + B} = \bar{A} \cdot \bar{B}$
- A NOR gate is equivalent to an AND gate with inverted inputs



A	B	$A + B$	$\bar{A}$	$\bar{B}$	$\overline{A + B}$	$\bar{A} \cdot \bar{B}$
0	0	0	1	1	1	1
0	1	1	1	0	0	0
1	0	1	0	1	0	0
1	1	1	0	0	0	0

LOGIC SIMPLIFICATION

- Given a complicated expression, it may be possible to reduce the number of gates needed to be used or eliminate inputs completely
- Smaller terms can be grouped up together when using the theorems to simplify – If consistent
- The negations in the theorems can be removed if consistency is maintained
- Apply one theorem at a time – or you may get lost
- Expressions can also be expanded
  - $A \cdot B \Rightarrow A \cdot B \cdot C + A \cdot B \cdot \bar{C}$ - reverse of Redundancy a

Example

- $Y = \bar{A} \cdot \bar{B} + \bar{A} \cdot B \cdot \bar{C} + \overline{A + \bar{C}}$

LOGIC SIMPLIFICATION

- $Y = \bar{A} \cdot \bar{B} + \bar{A} \cdot B \cdot \bar{C} + \overline{A + \bar{C}}$
- $Y = \bar{A} \cdot \bar{B} + \bar{A} \cdot B \cdot \bar{C} + \bar{A} \cdot C$ – De Morgan's Law on third term
- $Y = \bar{A} \cdot \bar{B} \cdot C + \bar{A} \cdot \bar{B} \cdot \bar{C} + \bar{A} \cdot B \cdot \bar{C} + \bar{A} \cdot B \cdot C + \bar{A} \cdot \bar{B} \cdot C$ – Reverse redundancy on the 2 input terms
- $Y = \bar{A} \cdot \bar{B} \cdot C + \bar{A} \cdot \bar{B} \cdot \bar{C} + \bar{A} \cdot B \cdot \bar{C} + \bar{A} \cdot B \cdot C$ – Idempotent, $\bar{A} \cdot \bar{B} \cdot C$ was doubled up
- $Y = \bar{A} \cdot C + \bar{A} \cdot \bar{B} \cdot \bar{C} + \bar{A} \cdot B \cdot \bar{C}$ - Redundancy on first and fourth term
- $Y = \bar{A} \cdot C + \bar{A} \cdot \bar{C}$ - Redundancy on second and third term
- $Y = \bar{A}$ - Redundancy

LOGIC SIMPLIFICATION – TRUTH TABLE

- $Y = \bar{A} \cdot \bar{B} + \bar{A} \cdot B \cdot \bar{C} + \overline{A + \bar{C}} = \bar{A}$?

A	B	C	$\bar{A}$	$\bar{B}$	$\bar{A} \cdot \bar{B}$	$\bar{C}$	$\bar{A} \cdot B$	$\bar{A} \cdot B \cdot \bar{C}$	$A + \bar{C}$	$\overline{A + \bar{C}}$	$\bar{A} \cdot \bar{B} + \bar{A} \cdot B \cdot \bar{C} + \overline{A \cdot \bar{C}}$	$\bar{A}$
0	0	0	1	1	1	1	0	0	1	0	1	1
0	0	1	1	1	1	0	0	0	0	1	1	1
0	1	0	1	0	0	1	1	1	1	0	1	1
0	1	1	1	0	0	0	0	0	0	1	1	1
1	0	0	0	1	0	1	0	0	1	0	0	0
1	0	1	0	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	1	0	0	1	0	0	0
1	1	1	0	0	0	0	0	0	1	0	0	0

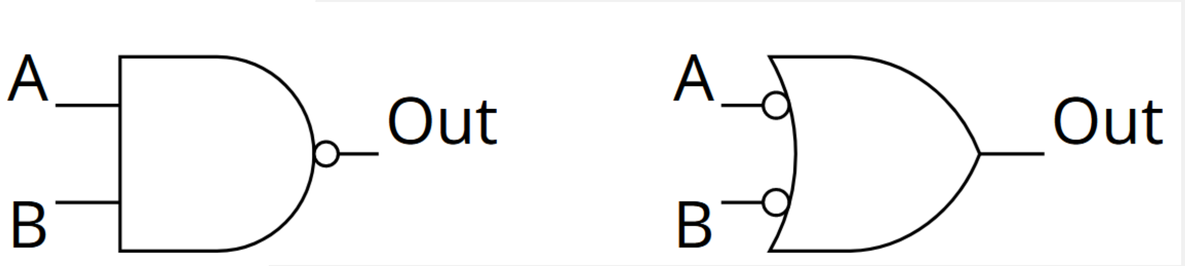
LOGIC SIMPLIFICATION

- Simplifying your expressions can greatly reduce the amount of hardware required to implement a function
- And reduce the number of variables you must keep track of
- The best solution varies – there is fewest gates, fastest, quickest to design, least cost, lowest power consumption
- You cannot get all of these at the same time – balancing act

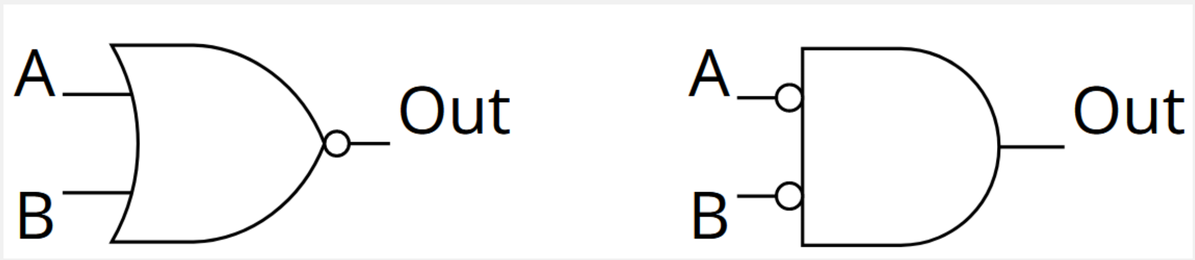
BUBBLE PUSHING

DE MORGAN'S LAW VISUALIZED

- Bubble pushing is a method of simplifying digital circuits
- Helps visualize how De Morgan's Law works
- The "bubbles" – NOT gates, can be shifted around the circuit if it remains consistent
- A bubble can be pushed through an AND or OR gate by changing the gate from OR to AND, or AND to OR



$$\overline{A \cdot B} = \bar{A} + \bar{B}$$

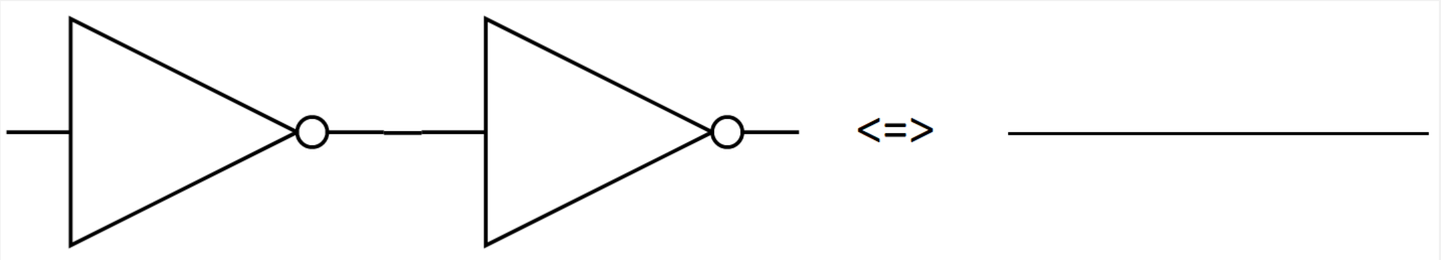


$$\overline{A + B} = \bar{A} \cdot \bar{B}$$

BUBBLE PUSHING

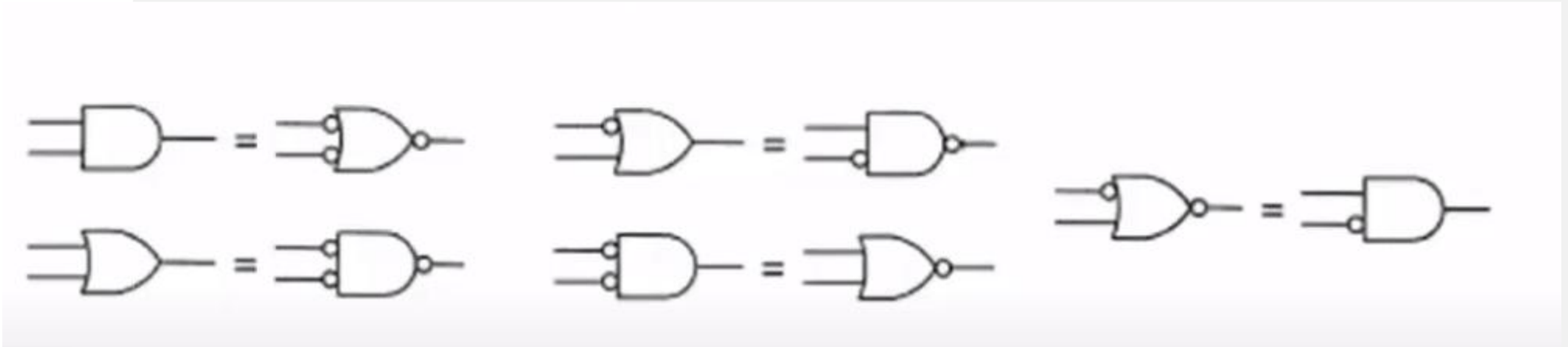
DE MORGAN'S LAW VISUALIZED

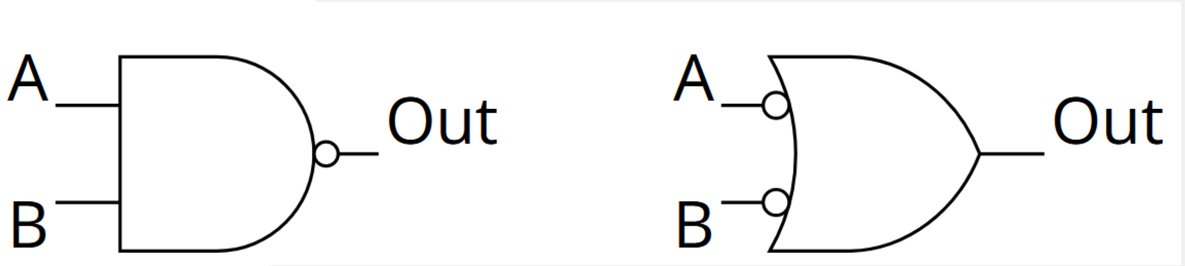
- If the bubble is pushed from the output to input, both inputs get a bubble
- Likewise, a bubble on both inputs are needed to push it to an output
  - You can add 2 inverters in a row before doing this to maintain consistency

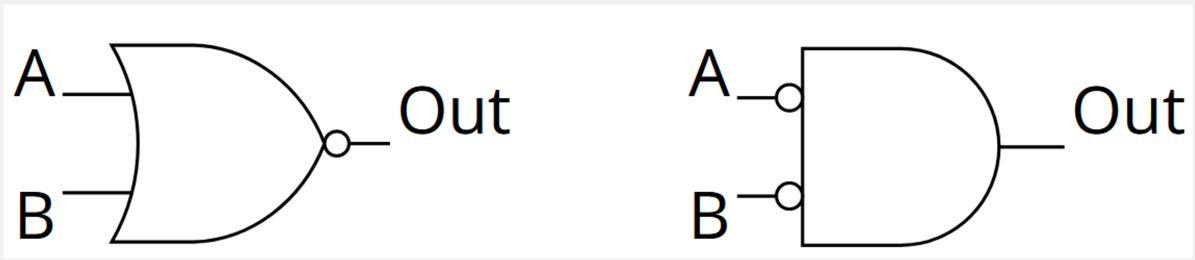


BUBBLE PUSHING

DE MORGAN'S LAW VISUALIZED







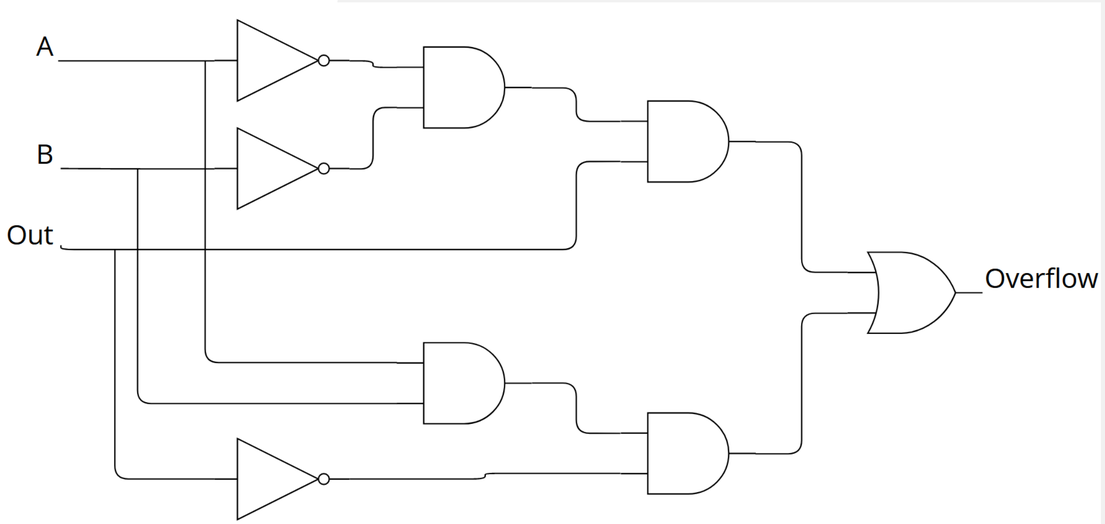
$$\overline{A \cdot B} = \bar{A} + \bar{B}$$

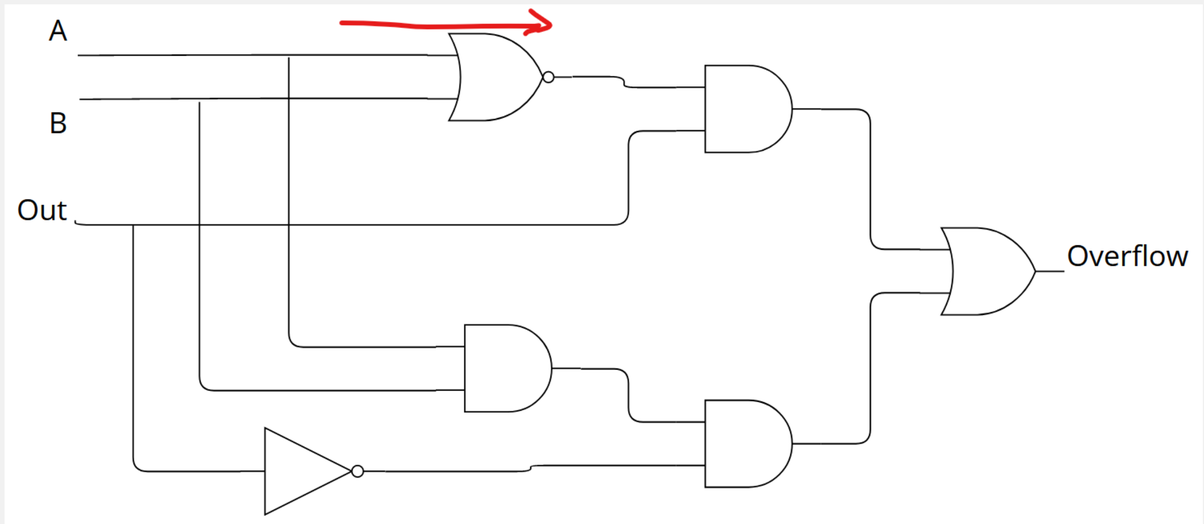
$$\overline{A + B} = \bar{A} \cdot \bar{B}$$

BUBBLE PUSHING

SIMPLIFY THE OVERFLOW CIRCUIT WITH BUBBLE PUSHING

- Aim to reduce the number of gates used

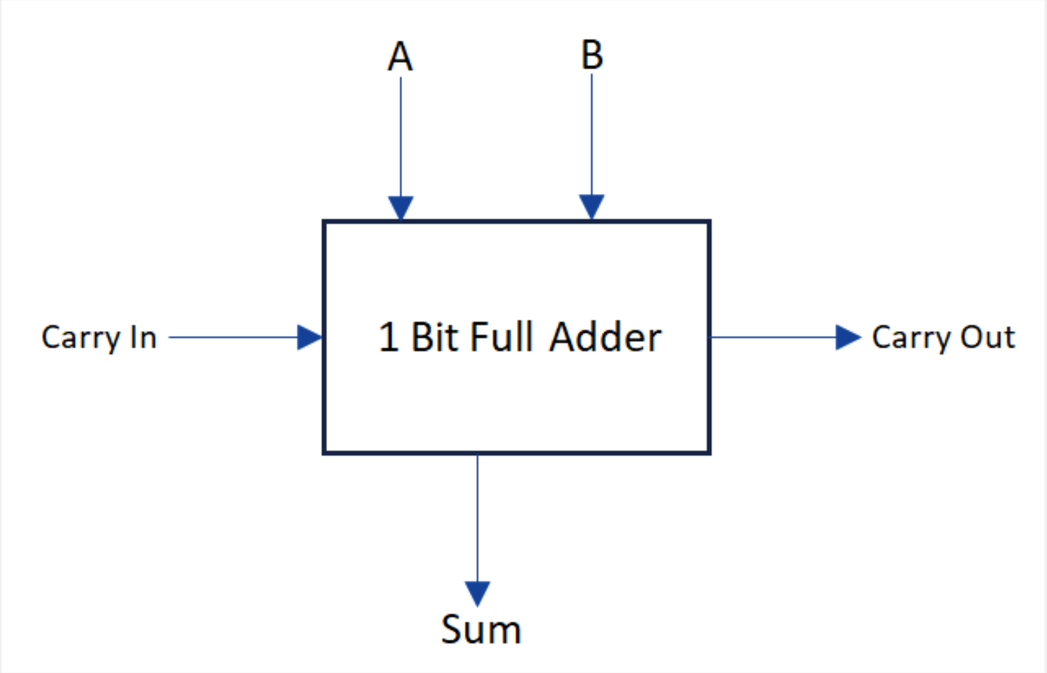




COMBINATIONAL BUILDING BLOCKS

ADDERS

- How computers performs addition of numbers
- Part of the Arithmetic Logic Unit (ALU)
- Full and half adders
- Consists of 3 inputs - half doesn’t have the carry in
  - A, B, Carry In
- 2 Outputs
  - Carry Out, Sum



COMBINATIONAL BUILDING BLOCKS

FULL ADDER TRUTH TABLE

Inputs			Outputs	
A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

COMBINATIONAL BUILDING BLOCKS

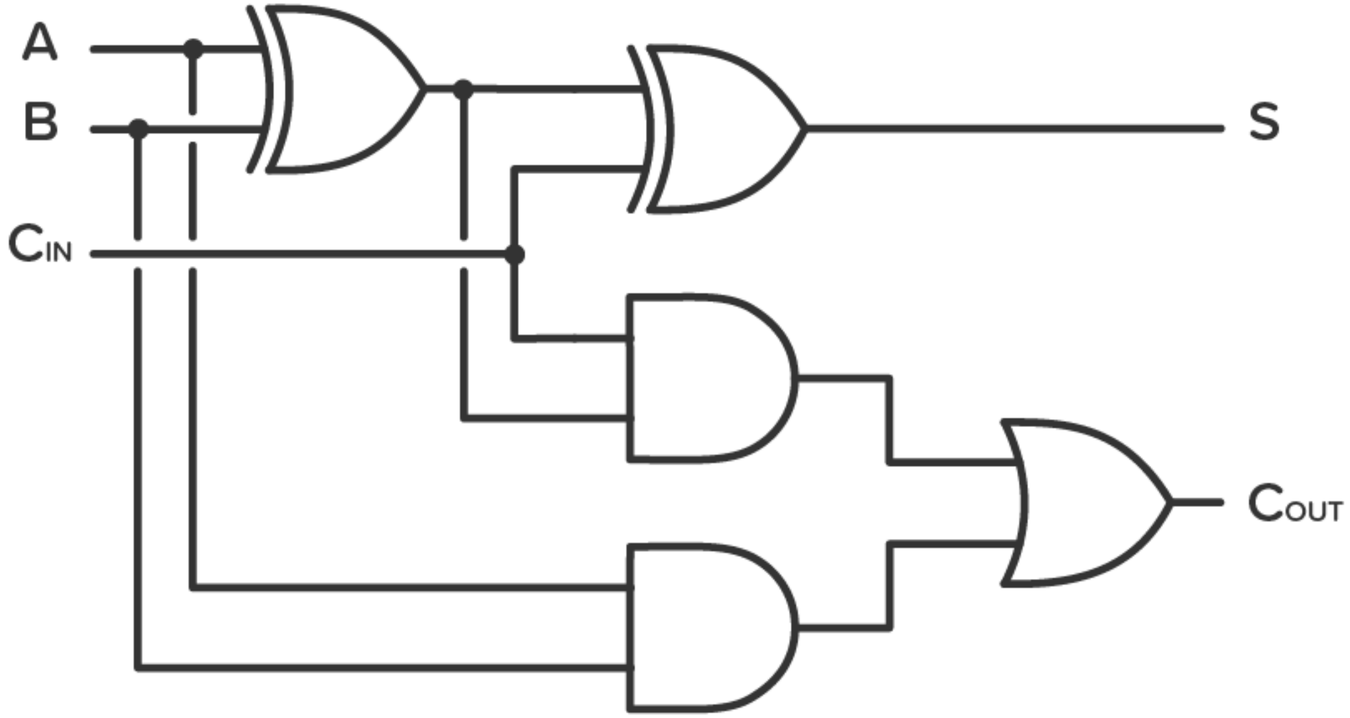
FULL ADDER BOOLEAN EXPRESSIONS

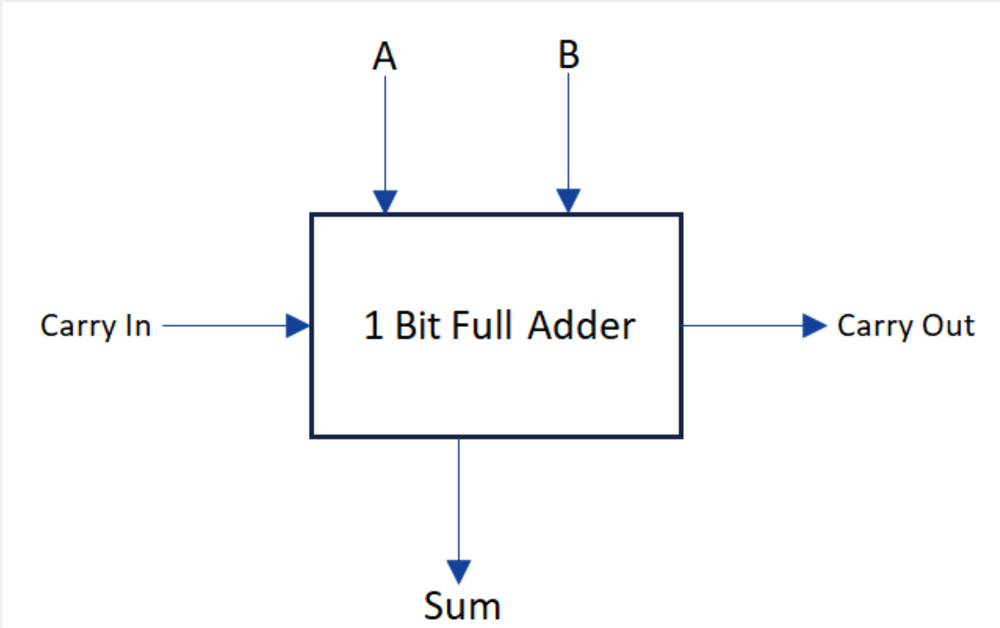
- $Sum = \bar{A} \cdot \bar{B} \cdot Cin + \bar{A} \cdot B \cdot \overline{Cin} + A \cdot \bar{B} \cdot \overline{Cin} + A \cdot B \cdot Cin$
- $Cout = \bar{A} \cdot B \cdot Cin + A \cdot \bar{B} \cdot Cin + A \cdot B \cdot \overline{Cin} + A \cdot B \cdot Cin$
- A bit messy, try and simplify it
- For Sum, there is an XOR with both the 2nd and 3rd terms and an XNOR with the 1st and 4th term
- $Sum = \overline{A \oplus B} \cdot Cin + A \oplus B \cdot \overline{Cin}$
- This is an XOR with Cin
- $Sum = A \oplus B \oplus$Cin

COMBINATIONAL BUILDING BLOCKS

FULL ADDER DIGITAL CIRCUIT

- *n* Full adders can be cascaded together to perform *n-bit* arithmetic operations

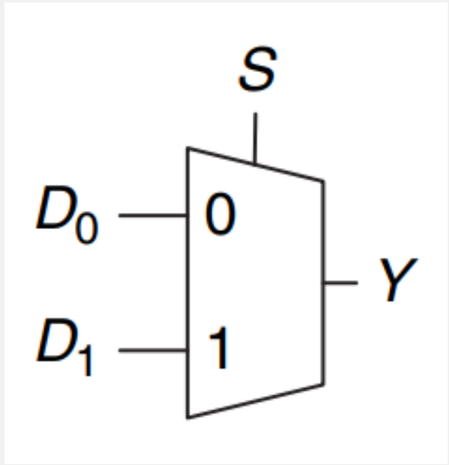




COMBINATIONAL BUILDING BLOCKS

MULTIPLEXER

- Used to select 1 input from multiple inputs
  - This input then flows through to the rest of the circuit
- Simplest is a 2:1 multiplexer
- Inputs – D0, D1 – data inputs, S – selector
- Outputs – Y – the data input allowed through



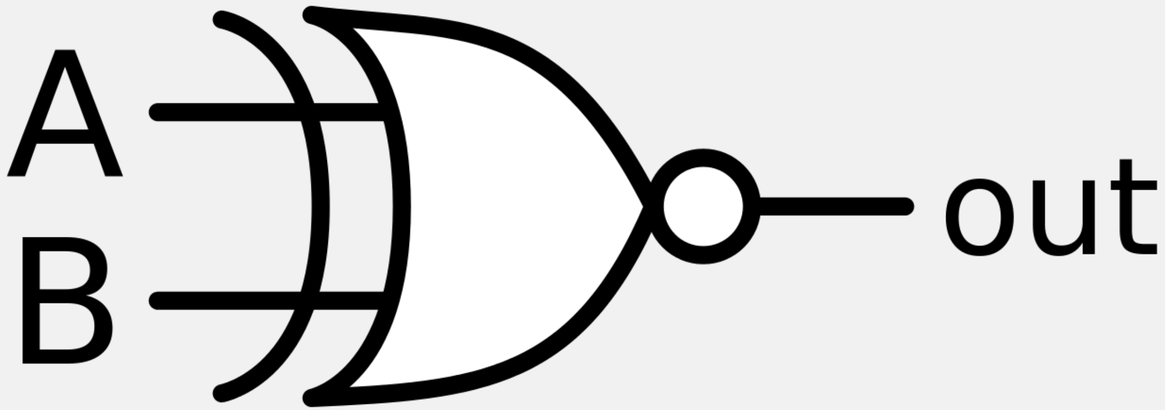
S	D0	D1	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

COMBINATIONAL BUILDING BLOCKS

COMPARATOR

- Takes two binary numbers A and B, then outputs whether they are equal, or if A < B or A > B
- Testing for equality is just an XNOR gate

A	B	A=B
0	0	1
0	1	0
1	0	0
1	1	1

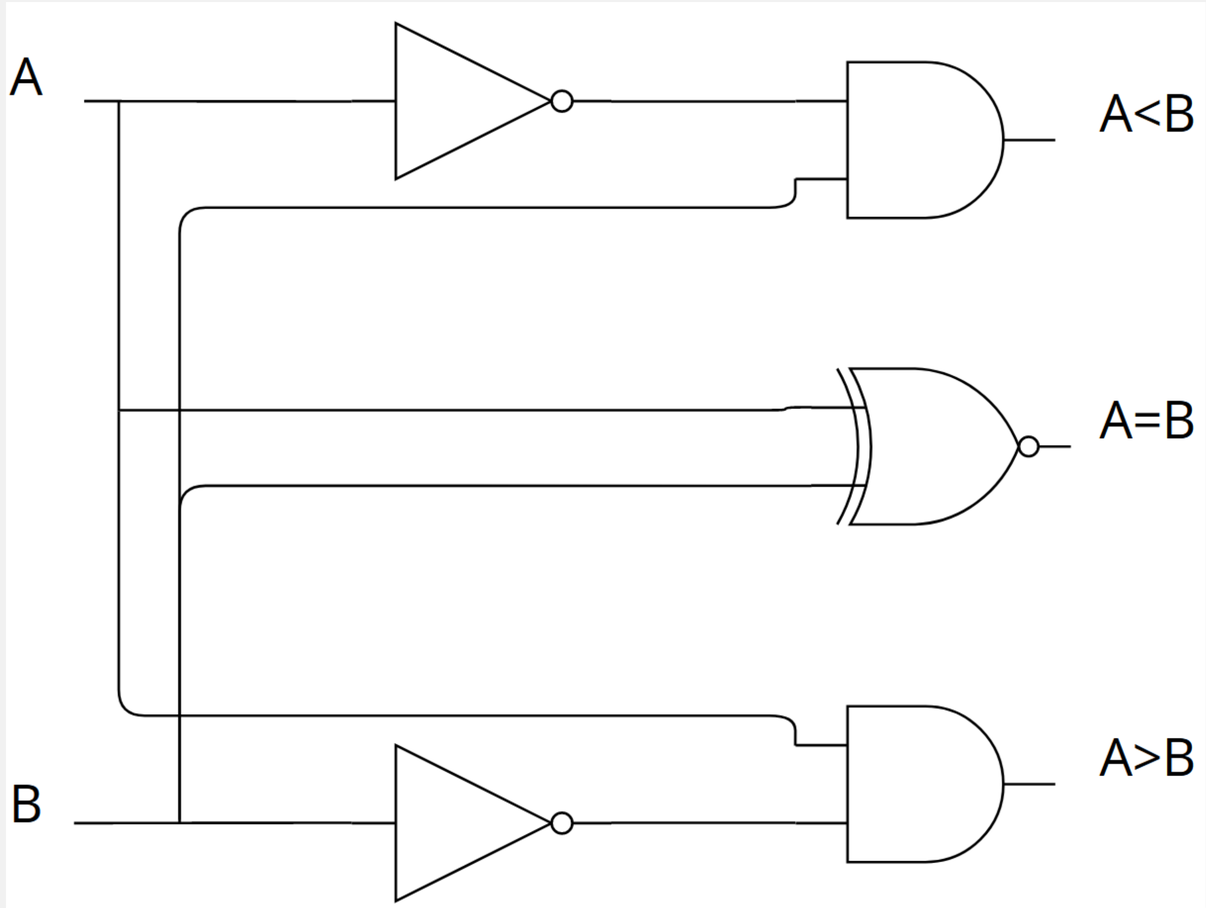


COMBINATIONAL BUILDING BLOCKS

COMPARATOR

- Testing for greater and less than is like an XOR gate
- The output for XOR is split into two separate outputs

A	B	A>B	A<B
0	0	0	0
0	1	0	1
1	0	1	0
1	1	0	0



SEQUENTIAL LOGIC

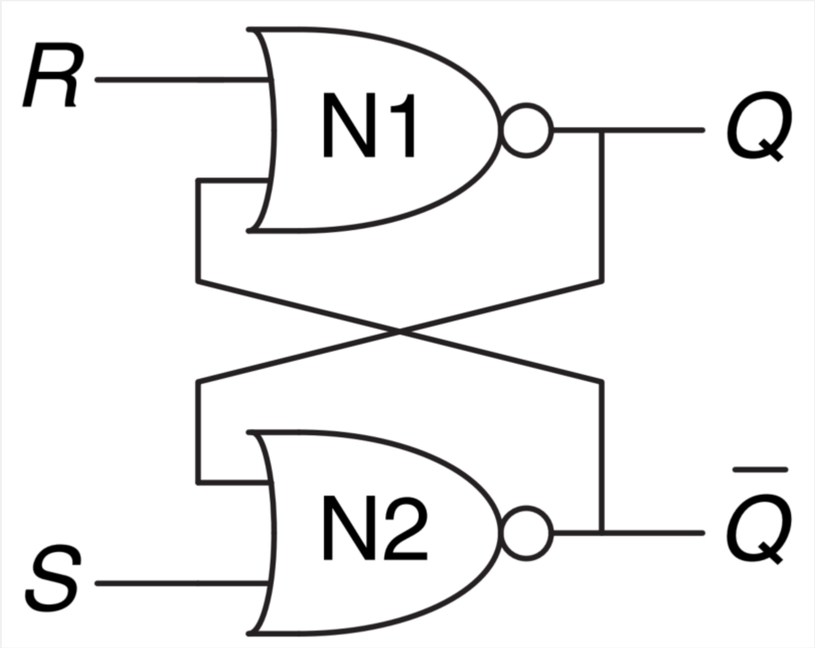
INTRODUCING STATE

- Outputs of combinational logic circuits only depended on the current state of the inputs
  - There were no loops
- Some operations may be dependent on an event that happened at an earlier point in time
  - A loop is formed when the output feeds back into an earlier part of the logic
- The state of the system is remembered

SR LATCH

BASIC MEMORY CIRCUIT

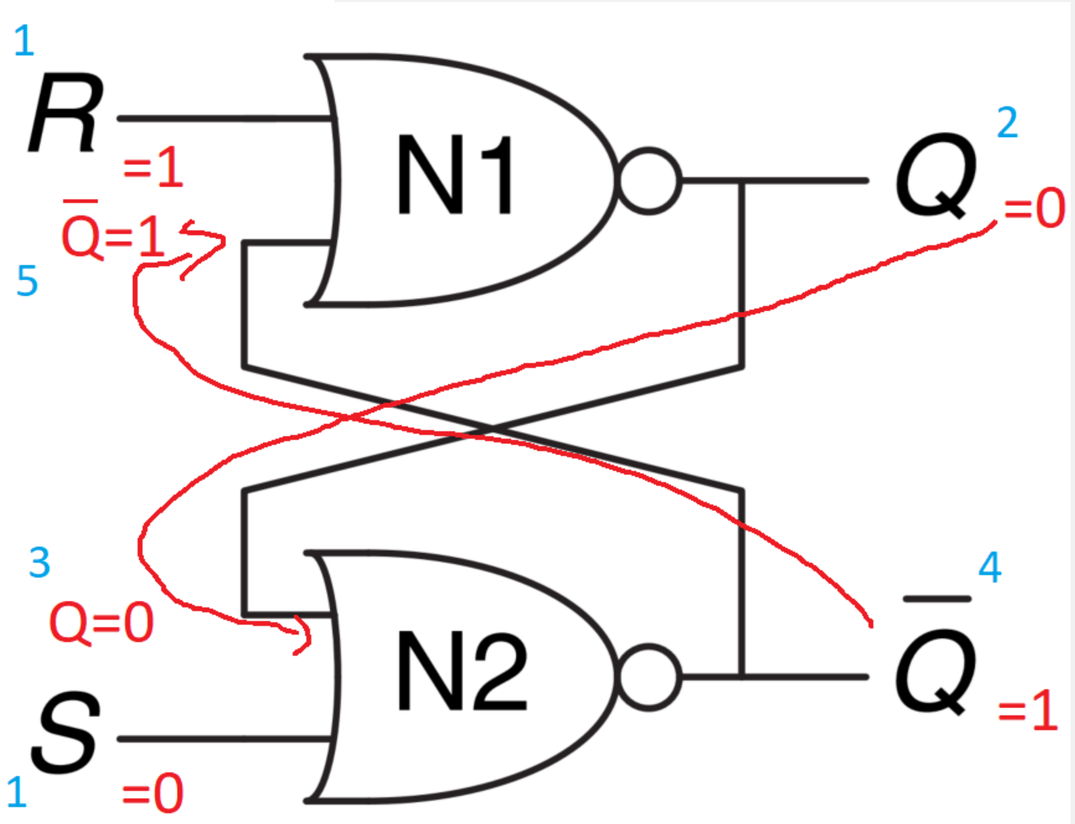
- 2 Inputs – Set and Reset
- 2 Outputs – Q and $\bar{Q}$ - complements of the same signal
- Consists of 2 NOR gates with the outputs looping back into the input of the other



SR LATCH

BUILDING THE TRUTH TABLE

- R = 1, S = 0
  - N1 sees at least 1 TRUE input (from R), so outputs a FALSE on Q
  - N2 sees both inputs as FALSE, so produces a TRUE on $\bar{Q}$



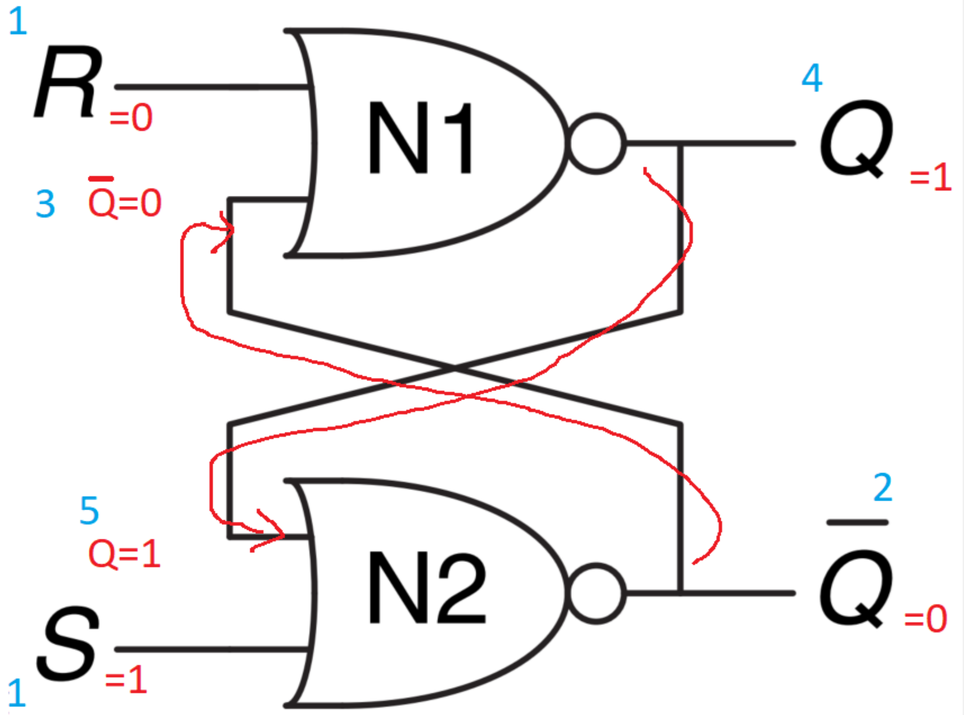
NOR Gate

R/S	$\bar{Q}/Q$	Out
0	0	1
0	1	0
1	0	0
1	1	0

SR LATCH

BUILDING THE TRUTH TABLE

- R = 0, S = 1
  - N1 sees 0 from R, but don't know what $\bar{Q}$ is yet
  - N2 sees a TRUE from S, so produces a FALSE on $\bar{Q}$
  - N1 now sees FALSE on both inputs and outputs TRUE on Q



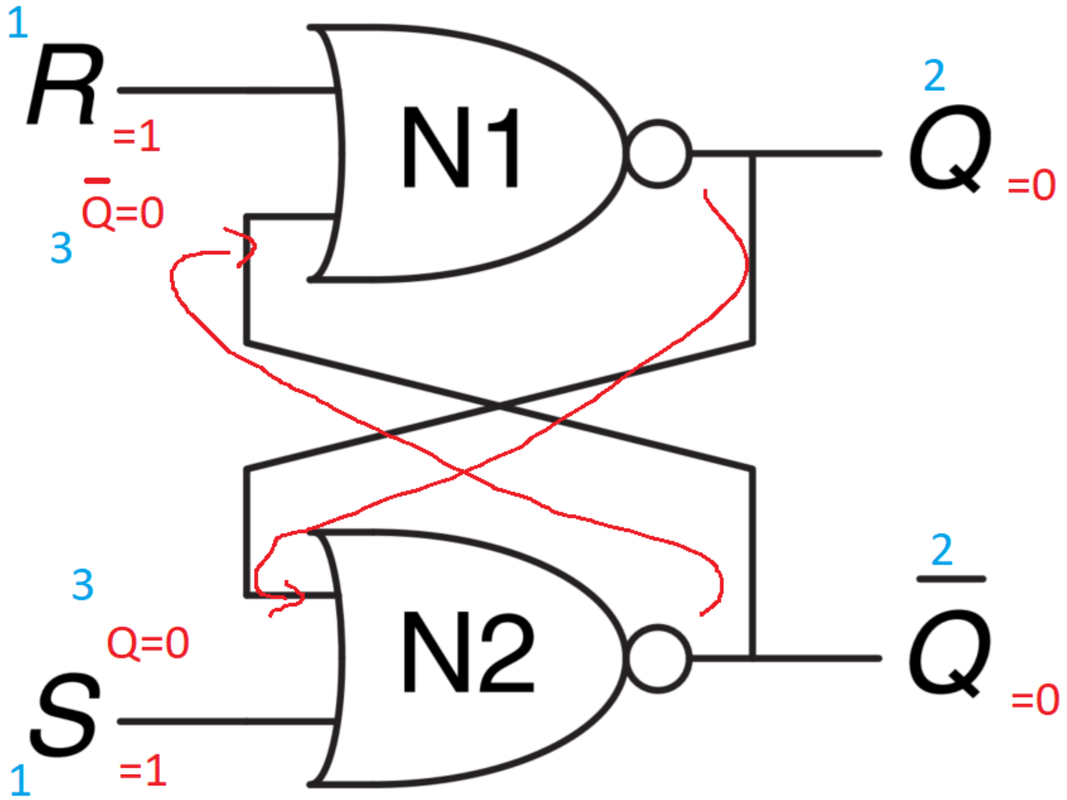
NOR Gate

R/S	$\bar{Q}/Q$	Out
0	0	1
0	1	0
1	0	0
1	1	0

SR LATCH

BUILDING THE TRUTH TABLE

- R = 1, S = 1
  - N1 and N2 both see a 1 so both Q and $\bar{Q}$ are FALSE
  - This is an invalid output since Q and $\bar{Q}$ are meant to be complements



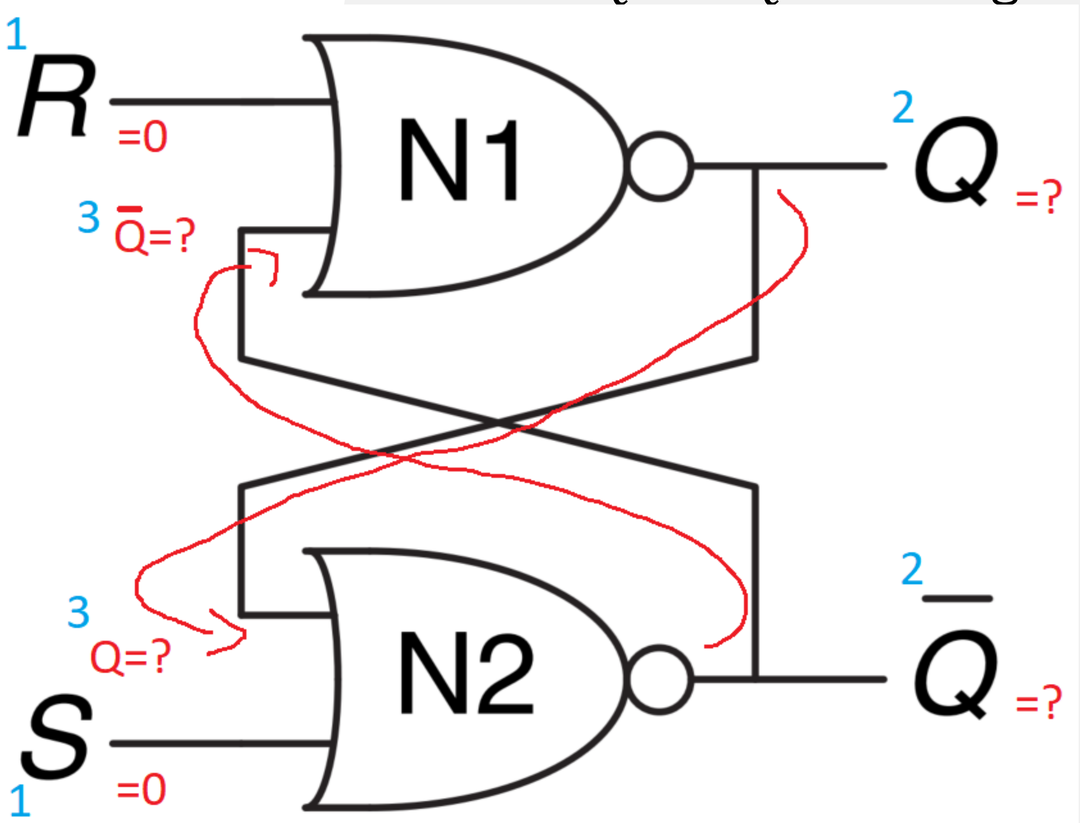
NOR Gate

R/S	$\bar{Q}/Q$	Out
0	0	1
0	1	0
1	0	0
1	1	0

SR LATCH

BUILDING THE TRUTH TABLE

- R = 0, S = 0
  - N1 and N2 both see a 0
  - We don't know what Q and $\bar{Q}$ are though



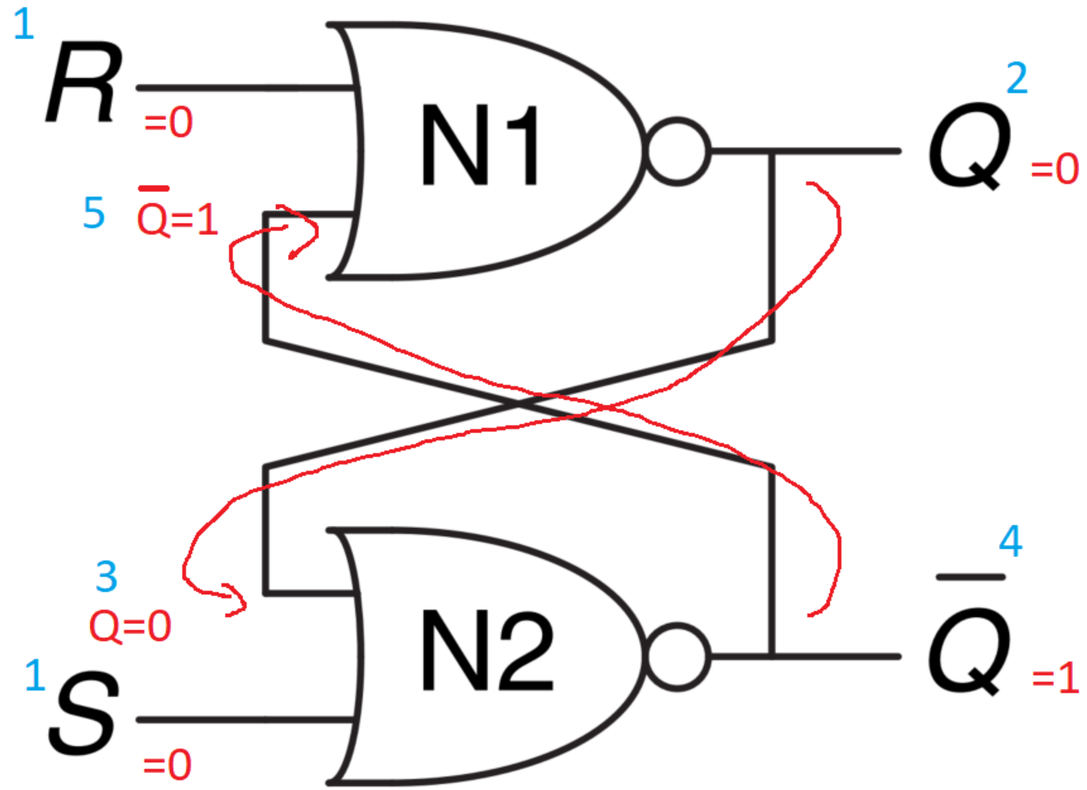
NOR Gate

R/S	$\bar{Q}/Q$	Out
0	0	1
0	1	0
1	0	0
1	1	0

SR LATCH

R=0, S=0 – CASE 1

- Assume $Q = 0$, then N2 sees both inputs as FALSE and outputs TRUE for $\bar{Q}$
- N1 then sees a 1 on an input and outputs FALSE
- Q is unchanged



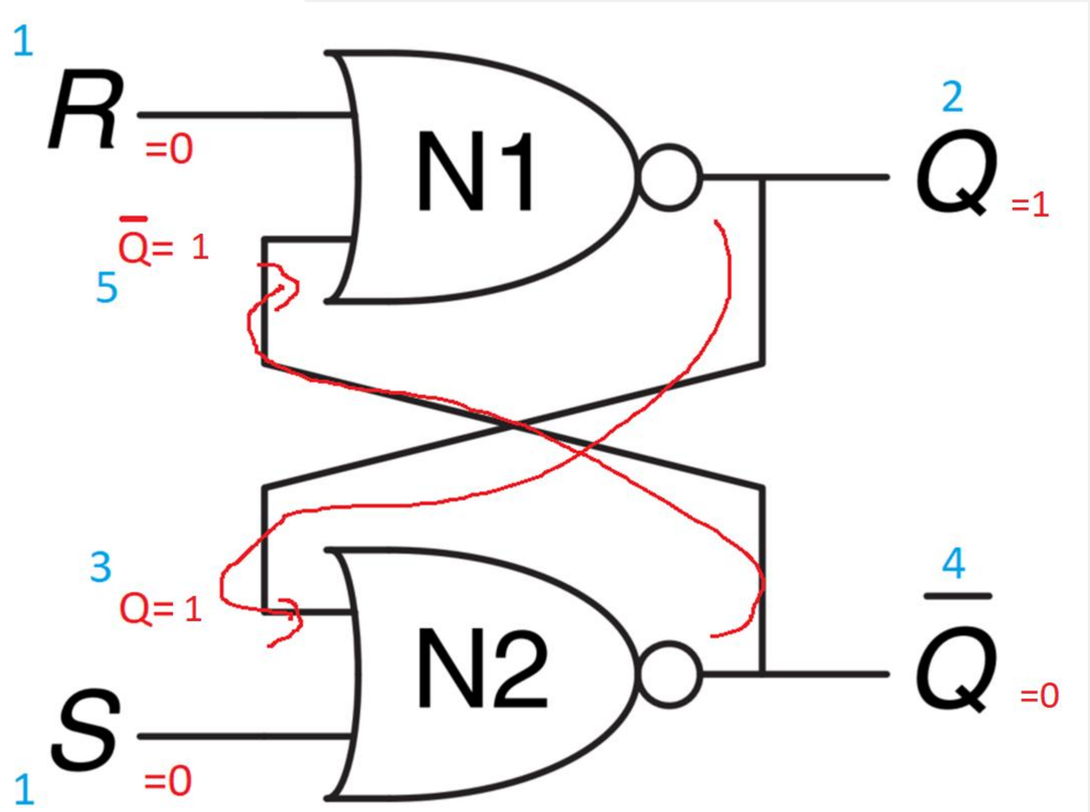
NOR Gate

R/S	$\bar{Q}/Q$	Out
0	0	1
0	1	0
1	0	0
1	1	0

SR LATCH

R=0, S=0 – CASE 2

- Assume $Q = 1$, then N2 sees one input as TRUE and outputs FALSE for $\bar{Q}$
- N1 then sees 0 on both inputs and outputs TRUE for Q
- Again, Q is unchanged



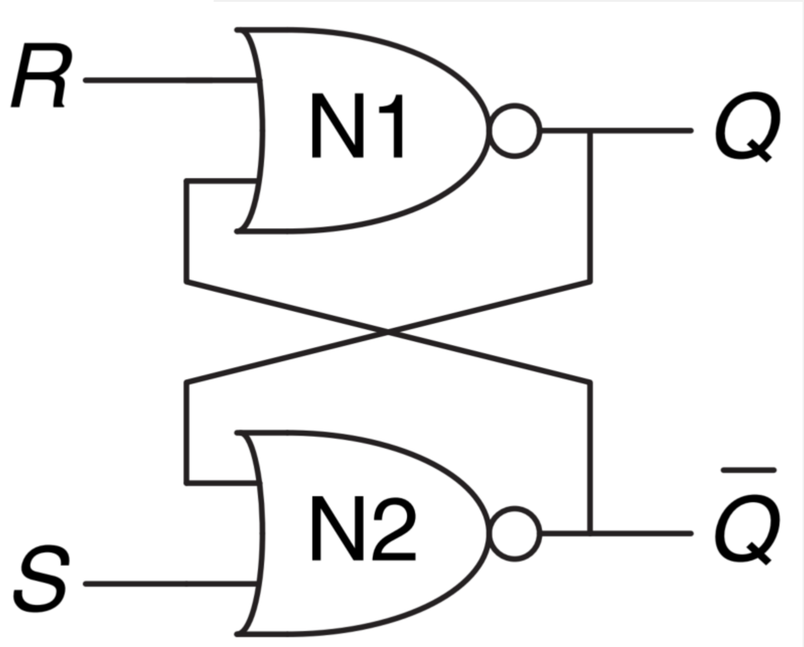
NOR Gate

R/S	$\bar{Q}/Q$	Out
0	0	1
0	1	0
1	0	0
1	1	0

SR LATCH

FULL TRUTH TABLE

- Reset = 1 makes the output Q = 0
- Set = 1 makes the output Q = 1
- Both set and reset being = 0 remembers the previous state
- Both set and reset being = 1 is an invalid state

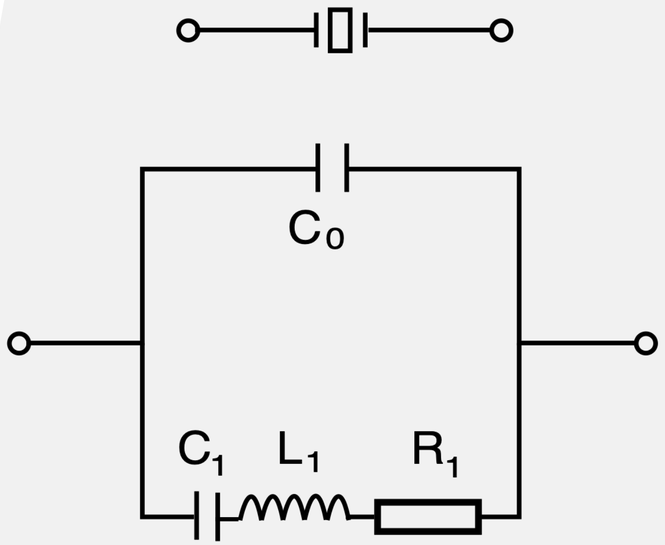


R	S	Q	$\bar{Q}$
0	0	Q_{prev}	$\bar{Q}_{prev}$
1	0	0	1
0	1	1	0
1	1	0	0

CLOCK

CLOCK

- A signal that operates at a set frequency – think CPU clock rate
  - 1 GHz = 1ns cycle time
- Can be sourced from an RLC circuit, quartz crystal
- Provides a way to control when events occur in a digital circuit in a predictable manner

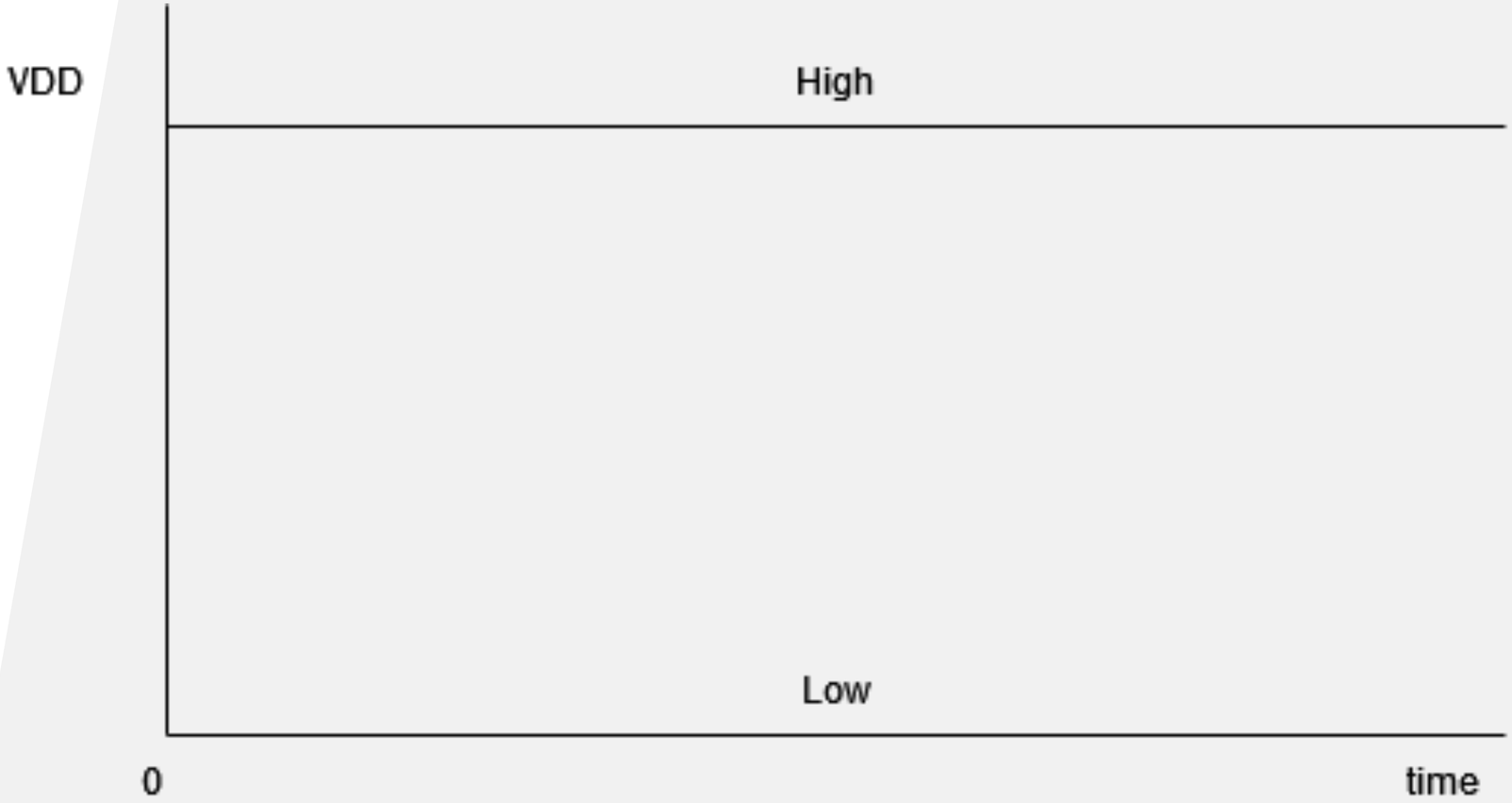




SIGNALS

HIGH AND LOW

- High = 1
- Low = 0



SIGNALS

RISING AND FALLING EDGE

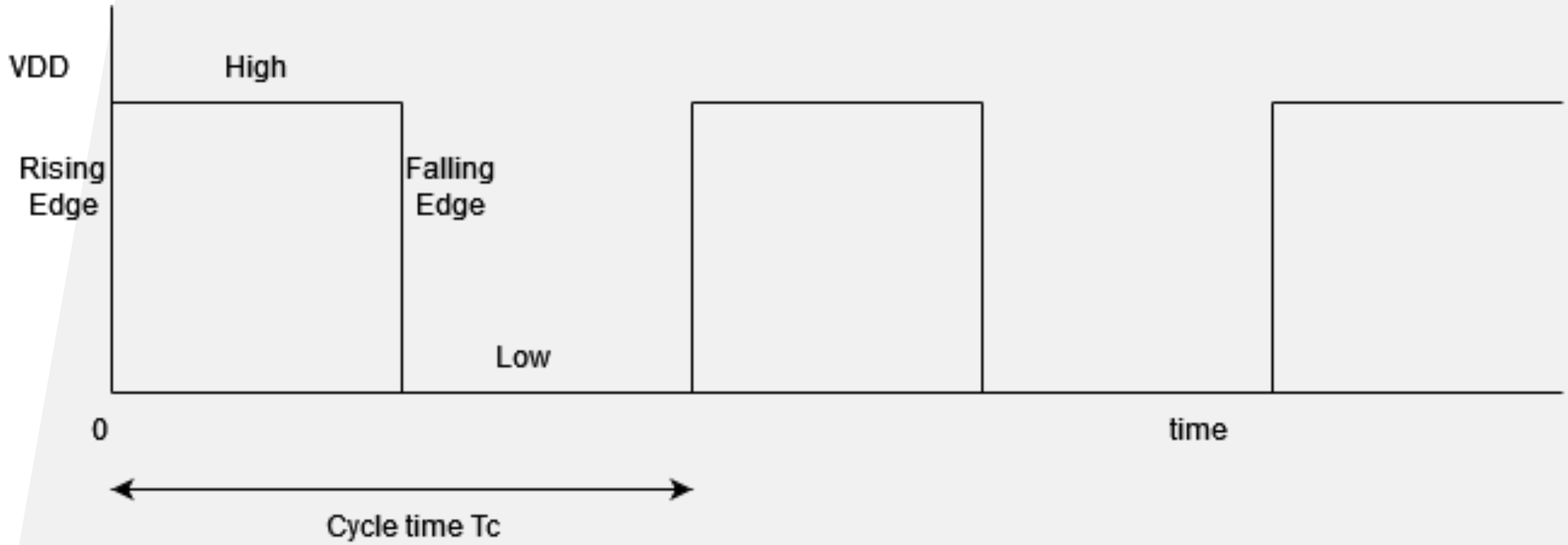
- Rising – Low to High
- Falling – High to Low
- Not a vertical (infinite slope) line in reality



SIGNALS

CLOCK SIGNAL

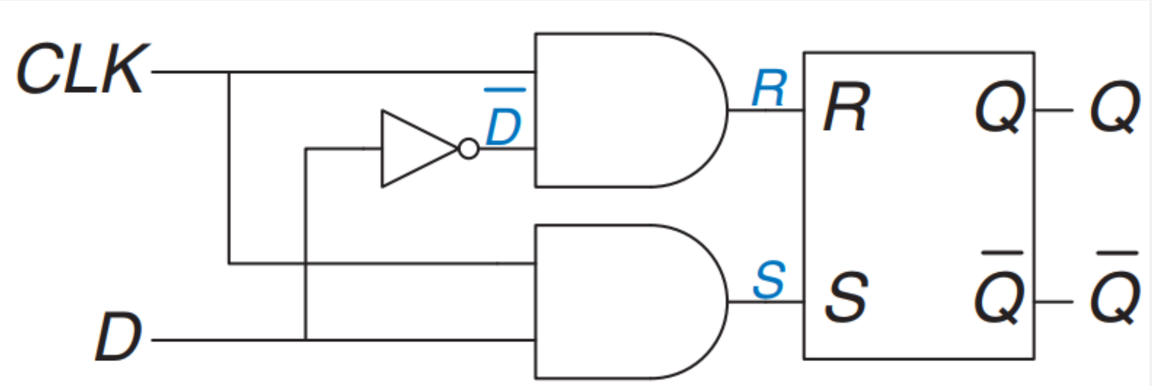
- Frequency $= \frac{1}{T_c}$



D LATCH

SOLVING THE R=1, S=1 PROBLEM

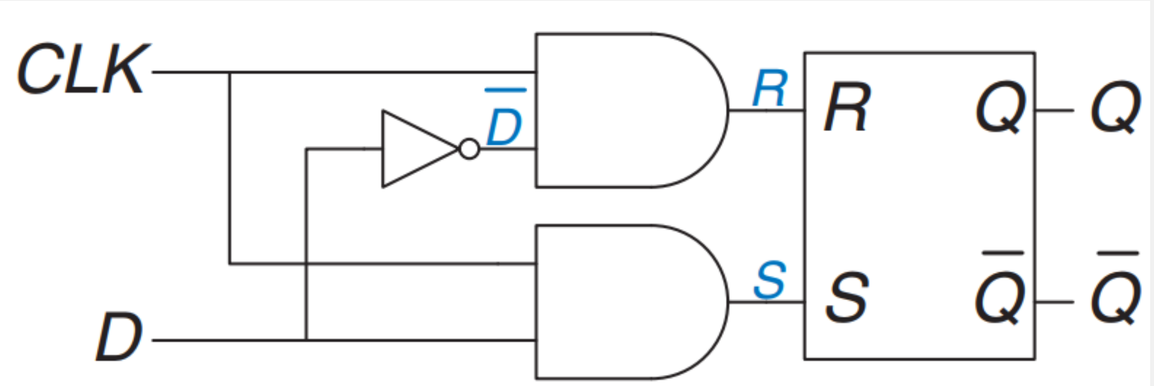
- The SR latch has the weakness of combining both ***what*** and ***when***
  - What the output should be
  - When it should happen – immediately
- Digital circuits become easier to understand when ***what*** and ***when*** are separated
- The D Latch solves this issue by introducing a clock signal (CLK) and a Data input (D)



D LATCH

OPERATION

- When CLK is 0, R and S are both 0 no matter what the value of D is
- When CLK is 1, D becomes S and $\bar{D}$ becomes R
- The D-latch then behaves the same way as the SR Latch
- By ANDing the CLK signal and $\bar{D}$, we ensure that R and S will never both be 1 at the same time

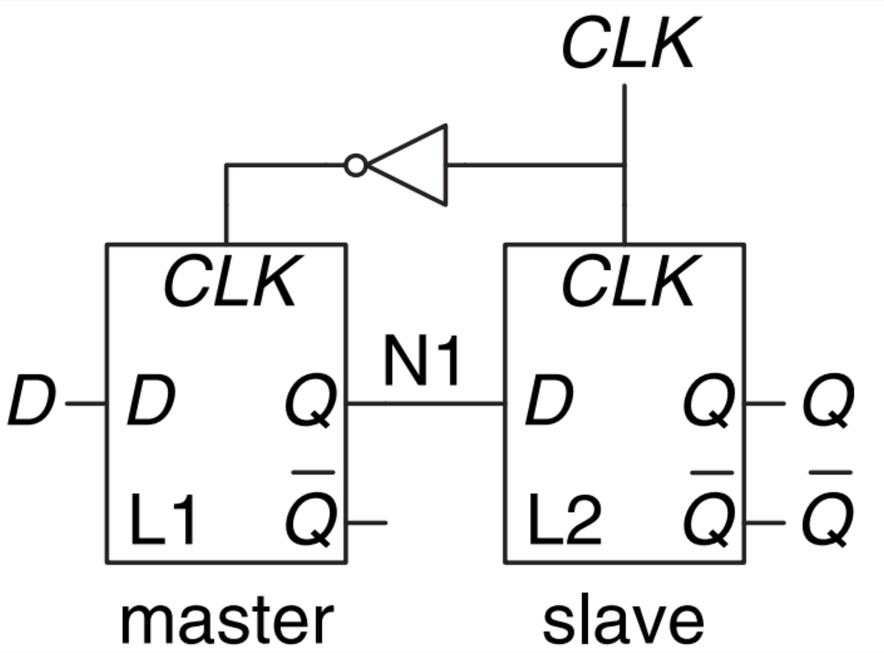


CLK	D	$\bar{D}$	R	S	Q	$\bar{Q}$
0	X	X	0	0	Q_{prev}	$\bar{Q}_{prev}$
1	0	1	1	0	0	1
1	1	0	0	1	1	0

D FLIP-FLOP

CONTROLLING WHEN D EFFECTS Q

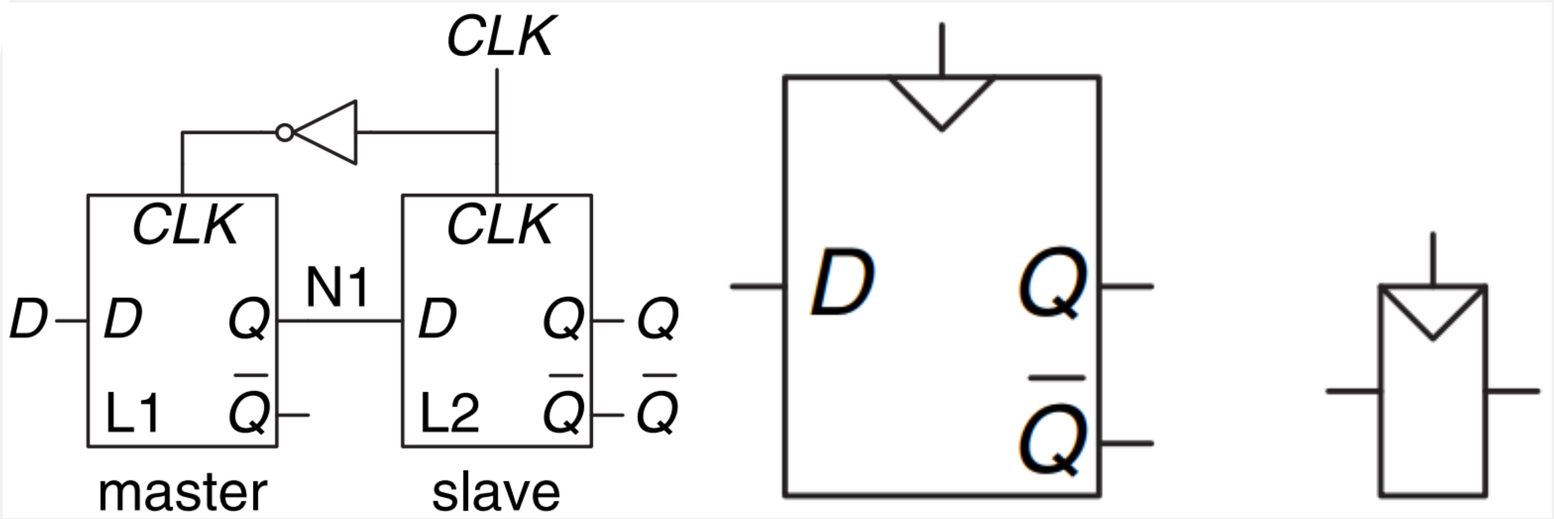
- The D-Latch will change the output Q whenever CLK is high
  - And may switch multiple times during this timeframe
- It would be better if we can control when data flows through
- Connect two D-Latches in series, one controlled by CLK and the other by $\overline{CLK}$



D FLIP-FLOP

Q ONLY CHANGES AT ONE POINT IN A CLOCK CYCLE

- When CLK is 0, master becomes *transparent* and allows D through and change N1, while slave is *opaque*
- When CLK changes to 1, master becomes opaque, and slave becomes transparent, while N1 is allowed through, and the output Q is set
- D is copied to Q on the *Rising Edge* of CLK and remembers its state at all other times



REGISTERS

STORING VALUES

- A group of Flip-Flops that share a common CLK input are called registers
- All bits are updated at the same time
- 4-bit register – store up to a value of 16

